Scalable Parallel-in-Time Integration for Equations of Motion

Nathan Chapman
Department of Computer Science, Central Washington University
August 15, 2025

Abstract

Physical simulations always need to balance accuracy and run-time. This work implements the Parareal Algorithm using graphics processing units across a distributed system to accurately simulate time-dependent physics while attempting to minimize runtime. Data-transfer latency is identified as the primary bottleneck, for which mitigation methods are provided. Benchmarks comparing single-threaded, single-GPU, and distributed implementations on a spectrum of coarse and fine discretizations are provided.

This work is made possible thanks to
my friends for sharing laughs and rants,
Mr. Chris Lacy for making physics phun,
Dr. Brandon Peden for showing me how to be a physicist,
and Dr. Andy Piacsek for never giving up on me.

I wouldn't have been able to do it without you.

Contents

1. Introduction **1**

2. Background **3**

3. The Parareal Algorithm **5**

3.1. Review of Equations of Motion 7

3.1.1. Initial Value Problems 7

3.1.2. Energy Drift & Symplectic Integrators 8

3.1.3. Iterative & Multiple-Shooting Methods 8

3.2. The Parareal Algorithm 9

3.2.1. Preparing the subproblems 9

3.2.2. Solving the subproblems 11

3.2.3. Solving the root problem 14

3.2.4. Converging the root solution 16

4. The Parareal Algorithm at Scale **18**

4.1. Review of High-Performance Computing 19

4.1.1. Multithreading & GPU Computing 19

4.1.2. Multiprocessing & Distributed Computing 21

4.2. Parareal on the GPU 23

4.3. Parareal on the Cluster 25

5. Performance Analysis **30**

5.1. Numerical Analysis 31

5.1.1. Discretization Error & Energy Drift 31

5.1.2. Stability 33

5.1.3. Convergence 35

5.2. Data Transfers & Latency 37

5.2.1. Host-Device Data Transfers 37

5.2.2. Host-Host Data Transfers 39

5.3. Benchmarks 41

6. Conclusion **46**

6.1. Future Work & Possible Optimizations 47

Bibliography ..... **I**

1. Introduction

Whether it is due to the amount of data that needs to be processed, or the accuracy needed in a simulation, the problems addressed in computational science can take significant time to solve via numerical methods i.e. the runtime. While this time has been reduced by improvements to the time needed to execute a single calculation, these improvements are decreasing. Another method to reduce the runtime is to increase the number of calculations that are done at the same time i.e. calculating in parallel. Though many problems in computational science simulate processes which evolve over time, which has traditionally been unable to be parallelized. The field of Parallel-in-Time Integration (PinT) allows the dynamics of a problem to be calculated in parallel, thus reducing the amount of time needed to solve the problem.

Even though PinT allows every dimension of problem to be calculated simultaneously, the speedup factor is still limited by how many calculations can happen simultaneously. A single central processing unit (CPU), at the time of writing, can execute about ten calculations at the same time. On the other hand, a single graphics processing unit (GPU) can execute about ten thousand calculations at the same time. For even further parallelization, calculations can be distributed over a cluster of machines, resulting in parallelism that is only limited by the size of the cluster. The use of GPUs and distributed systems of machines together offers a foundation on which an arbitrarily large problem could be solved in minimal time.

It's not uncommon for available hardware to provide more power than what's needed for a problem to only be spatially parallelized. For example, a simulation of a wave could discretize space to a degree such that any higher resolution would not provide a significant increase in accuracy, and to satisfy the Courant-Friedrichs-Lewy condition (CFL), time must be discretized to a similar degree. Modern high-performance computing (HPC) systems can execute all calculations for all spatial intervals simultaneously while having compute capability left over. Thus, in order to fully utilize these HPC systems, PinT methods must be not only used, but implemented to take advantage of the resources provided such as GPUs and distributed systems. That's where this work comes in.

This work focuses on implementing the Parareal algorithm (PA) from PinT to use HPC methods and resources in order to more efficiently simulate the dynamics of physical systems, in particular wave-like motion. While wave-like motion is the focus of the physics, this implementation and can be used to simulate other types of motion such as molecular dynamics, weather and climate forecasting, plasma dynamics in fusion reactors, or any system that is modeled by an equation of motion. With this work, computational modeling and simulation can scale not only with the needed accuracy of the problem but also with the performance and availability of hardware. Overall, HPC resources will be more efficiently utilized, results will be accurate as possible, and most importantly, time will be minimized.

This work is composed of five parts: an overview of the field of PinT, a presentation of the PA itself, a presentation of how the PA can be implemented to use HPC methods, and analyses on the performance of the implementation. Section 2 details the landscape in which this project lies

including other projects using high-performance implementations of PinT algorithms. Section 3 serves as a review of the PA from PinT in the context of numerically solving equations of motion while also presenting it in such a way that implementing it at scale is a natural extension. Section 4 serves as the core of this work describing the details of implementing the PA using high-performance methods. Section 5 provides analysis the performance of the implementation providing benchmarks, identifying sources of and suggesting methods to mitigate latency associated with transferring data, and traditional numerical analysis of results.

Section 2 gives an overview of the most notable methods and approaches used in PinT, how PinT has been used, and some examples of implementations of certain PinT algorithms. These notable methods include those based on spectral deferred corrections, multigrid reductions, and multiple-shooting. PinT has been used for real science ranging from simulating the blood flow in fish to gravitational collapse. Select implementations include those based on small-scale multiprocessing as well as full-scale supercomputing.

Section 3 first offers a review of concepts in computational physics that are fundamental to this work, and details how the PA can be used to simulate motion in parallel. Section 3.1 provides baseline knowledge of how initial-value problems model motion, how energy drift can be used to measure the error of a simulated physical system and how symplectic integrators can be used to mitigate this error, and two types of methods of which the PA can be considered an instance. Section 3.2 goes through the PA itself, presenting each key step in a way that makes the extension to using HPC methods intuitive. While the PA is not a new contribution, the presentation of it in this way is not only new but also key to understanding the details of why it is so well-suited for HPC.

Section 4 first presents a review of concepts in HPC that are fundamental to this work, and then how the PA can be implemented to take advantage of these HPC concepts. Section 4.1 focuses on reviewing the concepts of multithreading and how it applies to using GPUs for general-purpose computing, as well as multiprocessing and how it can distribute calculations over multiple machines. Section 4.2 first identifies how GPUs offer a meaningful increase in performance due to their incredible parallel-processing power and how to "simply move the expensive part to the GPU". Section 4.3 details how to construct and use a cluster of computers such that the PA can be executed on GPUs across multiple machines that are possibly not even in the same physical location.

Section 5 covers analysis of this implementation. Section 5.1 details the numerical effects of discretization on error/energy drift, stability, and convergence. Section 5.2 describes the influence, issues, and mitigation methods of transferring data between memory spaces. Section 5.3 addresses the core goal of this work: how this implementation affects the time needed to simulate motion.

2. Background

The PA has been one of the most widely studied PinT algorithms [1], but there are several other PinT methods that have garnered attention over the years (as seen in Figure 1). These methods and their uses have used several different implementations ranging from using small-scale, distributed systems to "true HPC" at Lawrence-Livermore National Lab (LLNL). These methods have also been used in practice for simulating the dynamics of systems from biology to gravitational collapse. This work aims to add to the field of PinT by building a foundation of using HPC methods for PinT in the scientific computing programming language Julia.

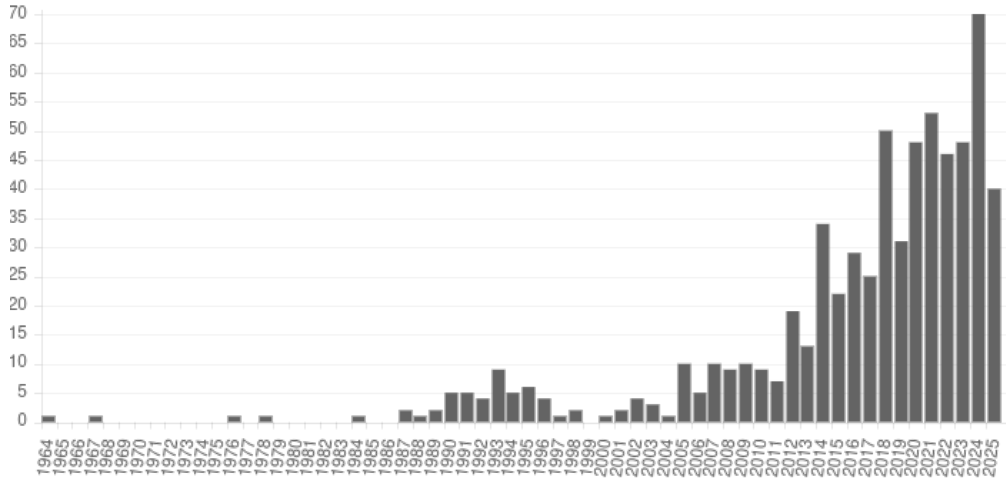

Figure 1: The number of papers published (shown on the vertical axis) regarding parallel-in-time integration has significantly, and steadily, increased over the past few decades. It is also worth noting that at the time of writing, the number of papers published this is year is on track to match last year's. Image credit [1]

While this work focuses on the PA, there are several other notable algorithms that have been developed. There is of course the PA [2], but the Parallel Implicit Time-Integrator (PITA) method has been developed as an implicit variation [3]. The Parallel Full Approximation Scheme in Space and Time (PFASST) [4], [5] and Revisionist Integral Deferred Correction (RIDC) [6] are based on the idea of deferred-corrections. A sub-class of PinT algorithms is based diagonalizing the time discretization matrix and decoupling an "all-at-once" system into a series of sub-systems [7]; this type of method is particularly notable because it is well suited for dissipative and hyperbolic problems [8]. On the other end, both the Space-time Multigrid (STMG), which treats the whole space-time domain simultaneously [9], and Space-time concurrent multigrid waveform relaxation (WRMG), which relies on cyclic reduction to run in polylog parallel time with linear serial complexity [10], [11], [12], [13], are well suited for parabolic partial differential equations. Finally, the Multigrid Reduction in Time (MGRIT) algorithm has been developed at Lawrence-Livermore National Lab to target hyperbolic problems, computational fluid dynamics, power grids, medical applications, etc. [14].

These algorithms have been implemented in various languages using different approaches of parallelism, though mostly OpenMP for CPU-based multithreading and MPI for multiprocessing. The PA has been implemented in Fortran as PararealF90 with versions using MPI and OpenMP [15], in Python using MPI [16], and in Julia using its native multiprocessing support [17]. PFASST has been implemented in C++ as PFASST++ [4], as well as in Python as pySDC using MPI [18], [19]. MGRIT has been implemented in Python as PyMGRIT using MPI [20], as well in C as XBraid also using MPI [21]. RIDC has been implemented in C++ as libridc [6] using OpenMP.

Between January and August of 2025 there have been 40 publications relating to PinT. Some of these investigations have applied these PinT methods to science and engineering problems such as: continuous-time optimal control problems [22], magnetohydrodynamics for plasma simulations in clean energy [23], formations of animal patterns in mathematical biology [24], and dynamics of financial markets with physics-informed neural networks [25].

Additionally, the aforementioned implementations have focused on using the traditional "work-horse" languages of HPC: C, C++, and Fortran. While these languages offer top-tier performance, scientists without expertise in them are unable to use their associated PinT implementations without first spending too much time learning the language. The Julia language was created to solve this "two language" problem with "the speed of C with the ease of Python" by using LLVM for just-in-time compilation and by being built from the ground up with high-performance scientific computing in mind. Julia seems to be the future of scientific computing, so there should be support for these PinT algorithms in it.

Needless to say, PinT methods have shown significant performance gains for a wide ranging collection of sciences. Because of this, it is paramount that PinT methods see continued support and implementation using high-performance methods and in scientist-focused programming languages. That's why this work provides an implementation of the PA using both massive multithreading on GPUs and scalable multiprocessing in the modern scientific computing language Julia.

3. The Parareal Algorithm

Simulating physical processes has traditionally been done sequentially; even during the modern age of hardware supporting parallel execution, using computers to calculate the evolution of physical phenomena has been sequential. Why haven't scientists just started doing things in parallel? Because of that pesky thing call *causality*; *the ball must go up before it can come down*. Because of this temporal dependence (spatial dependence has had its own workarounds such as the Barnes-Hut algorithm [26], [27]), simulation of large-time-scale physics has thus taken a long time to execute. The PA, and other parallel-in-time integration algorithms, have been developed in the last few decades to specifically address this issue [2]. Many details and variations of the PA have been investigated to find and address issues such as stability , convergence rates , application to higher-order differential equations . The main goal of this investigation is to contribute another variation: an implementation of the PA using methods from high-performance computing.

Before the PA can be implemented using these high-performance methods, the algorithm must be decomposed into its central components. The PA begins by partitioning a single IVP into several IVPs on smaller domains via an initial, inaccurate, "root" solution. Then each of the "subproblems" are solved using a sequential, accurate method on different threads at the same time. The final data for each of the subsolutions is then combined with the respective data of the root solution to yield a more accurate (i.e. "corrected") root solution. This new root solution is then used to repeat the process until convergence.

The interpretation of the PA in terms of these recursive subproblems makes the algorithm *almost* embarrassingly parallel; the corrections to the root solution need to be done sequentially. In addition to this structure, the algorithms being evaluated in parallel manifestly depend on simple arithmetic; because of this simplicity, the PA is well-suited to be evaluated on the GPU. Likewise, distributed methods can be combined with GPU evaluation for further parallelization for either a single model (taking advantaged of the recursive nature of the PA) or a system of models.

This chapter begins by casting the PA in a form that is conducive to being scaled. The main idea is to recast the "magical" mechanism of the PA to something that can be executed recursively. In other words, the PA takes an IVP and produces a collection of IVPs, which can each be given to another instance of the PA. The latter half of of this chapter is devoted to presenting a model for which the scalable version of the PA can be implemented. This model includes how the performance of the PA can increased by using a GPU on a single machine, as well as how to build and use a cluster of machines to further increase performance.

Example: To help understand and clarify the mechanisms of the scalable PA, including the important details that are not explicitly covered in the algorithm itself, an example problem is used. Consider the motion of a thrown ball just after it leaves the hand over the course of 8 seconds (ignoring air resistance). This motion is modeled by: $P = \left\{ \underbrace{\partial_t^2 \vec{r} = \vec{g}}_{\text{Acceleration}}, \quad \underbrace{\vec{r}(0) = \vec{r}_0,\ \vec{v}(0) = \vec{v}_0}_{\text{Initial values}}, \quad \underbrace{[0\text{s}, 8\text{s}]}_{\text{time span}} \right\}$. The machine has 8 cores.

Physically, **equations of motion** (EOM) (section Section 3.1) are considered in this work to be second order differential equations describing the motion of objects. Another way of interpreting an EOM is as how the acceleration of an object over time depends on the object's position and velocity at that time. The solution to an EOM is simply the position of the object as a function of time, from which the velocity can be derived. The EOMs alone though only provide the behavior of how the position and velocity of the object *changes* over time. In order to uniquely define a path the object takes, initial values for the position and velocity must be stipulated. The EOM together with these initial values, then define an **initial value problem** (IVP). These IVPs have long been studied, but investigations into physics at the most extreme scale have required significantly more resources.

3.1. Review of Equations of Motion

According to classical mechanics, the motion for any and every object in the universe can be determined for all time using only its current position, current velocity, and the forces acting on it [28]. The foundation on which this principle lies is the *equation of motion* (e.g. Newton's Second Law), which dictates how motion changes in time, or both time and space. These EOMs can be solved via numerical methods, but some methods better suited for specific problems than others, especially when considering the tradeoff between the accuracy of the result and the level of approximation. Additionally, some straight-forward methods solve the EOM by accurately stepping through time, but others first make an estimate of the solution, somehow make corrections to that guess, and then keep doing that until the desired accuracy is achieved.

3.1.1. Initial Value Problems

Take, for example, the motion of a simple pendulum. While the overall motion of a bob is determined from length on which it hangs, and the gravity affecting it, the angle at which the bob hangs only depends on time. Now, the position of a point on a guitar string does not only change in time, but is also affected by the motion of the points around it. Both of these systems can be modeled by an EOM, but the pendulum can be modeled an **ordinary differential equation** (ODE), and the guitar string can be modeled by a **partial differential equation** (PDE).

The nuances on each of these concepts are out of the scope of this work, but the detail that is indeed important here is that there are techniques that can transform a PDE to a collection of ODEs. One such procedure is known as the *spectral method*, where by representing the solution to the PDE as a sum of waves (i.e. a Fourier transform), the physics in each dimension only affects the frequency in that dimension [29]. While the solutions to the ODEs would be in so-called "frequency space", applying the inverse Fourier transform on those solutions, achieves the desired solution to the original PDE. Whether it be an ODE, a PDE, or a system of ODEs, when modeling physical phenomena, initial values need to be considered to make any concrete predictions about the future state of a specific object.

For our purposes, an IVP can be thought of as an object with several properties: the acceleration, the initial position and velocity, and the time interval on which you are modeling (which could be unbound e.g. $[0, \infty)$) as shown by the following equation:

$$\left\{ \underbrace{\partial_t^2 \vec{r} = \vec{F}(t, \vec{r}, \vec{v})}_{\text{Acceleration}}, \quad \underbrace{\vec{r}(0) = \vec{r}_0,\ \vec{v}(0) = \vec{v}_0}_{\text{Initial values}}, \quad \underbrace{[0, t_f]}_{\text{time span}} \right\}. \tag{1}$$

In some sense, anything that fits this structure, could be considered an IVP (more on this in Section 3.2). Because initial values "lock in" the trajectory of an object based on the EOM's physics, the IVP can be solved numerically.

3.1.2. Energy Drift & Symplectic Integrators

When it comes to solving EOMs numerically, possibly the biggest factor that should influence the choice of integration method is the length of time on which the EOM is considered. Almost all methods are not inappropriate to use on small scale problems, but some of these methods result in inaccurate or even unphysical behavior due to the accumulation of approximation-error. For EOMs, one way to quantify this error is through the idea of **energy drift**.

Outside of considering the energy of the whole universe, the total energy of a (well-defined) system does not change in time. If a system is composed on multiple objects, then the total energy of each object individually can change, but the total energy of all the objects combined is constant. This simple idea provides a reference with which to compare the simulated energy.

The difference between the simulated energy at any point in time and the initial energy acts as a measurement of the inaccuracy of the simulation at that point in time i.e. the *local error*. A corollary to this is that the difference between the energy at the end of the simulation and the initial energy serves as a measure of the *global error* as the local error compounds over time. In other words, the energy of the system *drifts* away from the true value as error is compounded.

While almost all methods *can* be used for any problem, some methods result in less energy drift for the same time-step. A specific class of these methods is known as **symplectic integrators**. The underlying reasons for this are out of the scope of this work, but an important note is that even though these methods mitigate the effects of energy drift substantially, they do not yield exactly zero drift [30]. It is these symplectic integration methods that are considered in this work.

3.1.3. Iterative & Multiple-Shooting Methods

When it comes to simulating physics that only depends on spatial behavior, certain methods can be used that aren't immediately available in the time-dependent case. Two classes of these methods are *iterative* and *multiple-shooting* methods. An **iterative method** can be considered a form of "guess and check" algorithm where an initial solution is given, some quantity is calculated using this solution, and then the process repeats after adjusting the solution to potentially result in a "better" calculated quantity. A **multiple-shooting method** is when one big problem is divided into many problems, each of which only considers a subset of the original domain, and each of these problems is solved independently such that the its values at the domain boundaries agree with those of its neighbors.

An iterative method known as the Hartree-Fock algorithm (also known as the Self-Consistent Field Method) can be used to find the configuration of atoms and molecules that minimizes the system's quantum energy [31]. Likewise, multiple-shooting methods have been used for solving optimal control problems [32]. While these methods have traditionally been used for spatial physics, this work relies on the combination of these principles to solve time-dependent problems.

3.2. The Parareal Algorithm

The main idea of the PA is to break up a single IVP into many smaller IVPs using some low-accuracy solution, solve those in parallel using high-accuracy methods, correct your initial solution using the sub-solutions, then make a new low-accuracy solution based on the corrected data, and repeat this process until the solution doesn't change. The end result of this procedure is a solution identical to one produced by directly using the high-accuracy method while potentially taking a less time [33]. Because the PA wraps traditional (sequential) solvers, it could be considered a "meta-" or "higher-order" method to solve IVPs.

3.2.1. Preparing the subproblems

Let the second-order initial value problem P be defined such that

$$P = \{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_0, \partial_t u(0) = v_0, \quad [t_0, t_0 + \Delta t]\}, \tag{2}$$

where $\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t)$ defines a second-order differential equation, $u(0) = u_0$, and $\partial_t u(0) = v_0$ are the initial conditions on the position and velocity, respectively, and $D = [t_0, t_0 + \Delta t]$ is the closed time interval from t_0 to $t_0 + \Delta t$. The discretization N of P should be determined by the number of available threads N_t such that $N = mN_t$, for some positive integer m. The discretization should be chosen in this manner for maximum performance and efficiency; if $N = mN_t + r$, and $0 < r < N_t$, each thread will solve a subproblem m times until on the $m + 1$ iteration where only r threads would be active while $N_t - r$ threads idle (assuming all threads are synchronized). This type of optimization is sometimes referred to as "*flooding the threadpool*" to mitigate *thread starvation* .

With the discretization decided, partition the time domain D into subdomains D_p such that

$$D_p = \left[t_0 + \frac{p}{N}\Delta t, t_0 + \frac{p+1}{N}\Delta t\right] = \left[t_p, t_{p+1}\right]. \tag{3}$$

Use a fast integration method $\mathcal{G}_0$ (such as the Euler method) to compute initial root solutions $\left\{u_p^0\right\}_p$, $\left\{v_p^0\right\}_p$ defined such that

$$\left\{u_p^0\right\}_p = \{u_0^0, u_1^0, u_2^0, ..., u_{N-1}^0\} \tag{4}$$

$$\left\{v_p^0\right\}_p = \{v_0^0, v_1^0, v_2^0, ..., v_{N-1}^0\} \tag{5}$$

and $\left\{u_p^0, v_p^0\right\} = \mathcal{G}(\Delta t, u_{p-1}^0, v_{p-1}^0, \partial_t^2 u)$.

Subproblems P_p take the same from as in Eq. 2 (this also means subproblems are themselves, problems), but instead using initial conditions defined by those in the initial root solution. For example, the subproblem P_1 for the second ($p = 1$) subdomain, takes the form

$$P_1 = \left\{\mathcal{L}(t, u, \partial_t u, \partial_t^2 u) = f(t), \quad u(0) = u_1^0, \partial_t u(0) = v_1^0, \quad \left[t_p, t_{p+1}\right]\right\}. \tag{6}$$

Algorithm 1 shows the pseudocode of this process for a given IVP and integration algorithm i.e. "propagator", resulting in root solutions and and the collected subproblems. With these subproblems in hand, the PA continues to its next stage: propagating these problems in parallel.

Algorithm 1: PREPARE THE SUBPROBLEMS

INPUT: Second order initial value problem `P`, Coarse solver `C`

OUTPUT: Solution for `P` made from `pos_seq` and `vel_seq`, and array of subproblems `subproblems`

// choose discretization

`N` ← number of subproblems *// e.g. multiple of # of computer cores*

// partition the domain of P into N subdomains e.g. [a, b] -> {[a, c], [c, b]}

`subdomains` ← `partition(P.domain)`

// use coarse solver to get positions and velocities for the root problem

`pos_seq`, `vel_seq` ← `propagate(P, C)`

// create subproblems

for `i` from 1 to `N - 1`

- `subdomain` ← `i`-th domain partition `subdomains[i]`
- `pos0` ← initial position for `i`-th subproblem `pos_seq[i]`
- `vel0` ← initial velocity for `i`-th subproblem `vel_seq[i]`
- `subproblems[i]` ← ivp on `subdomain` with initial values `pos0` and `vel0` for acceleration `P.acc`

return Solution for root problem with `pos_seq` and `vel_seq`, and array of subproblems `subproblems`

Algorithm 1: Prepare the subproblems

Example: Given the example IVP (Eq. 6) and the available threads,

1) There should be 8 subproblems because there are 8 available threads.
2) Create the 8 time sub-domains $[0, 1], [1, 2], ..., [7, 8]$.
3) Use the initial position and velocity to quickly solve the root problem via the Euler method to give a sequence of 8 total positions $\vec{r}_p^0$ and a sequence of 8 total velocities $\vec{v}_p^0$.
4) Use the calculated positions and velocities as initial positions and velocities to create 8 subproblems on the associated subdomains following the form

$$P_p = \left\{\partial_t^2 \vec{r} = \vec{g}, \quad \vec{r}(0) = \vec{r}_p^0,\ \vec{v}(0) = \vec{v}_p^0, \quad \left[t_p, t_{p+1}\right]\right\}. \tag{7}$$

The result of this process is shown in Figure 2.

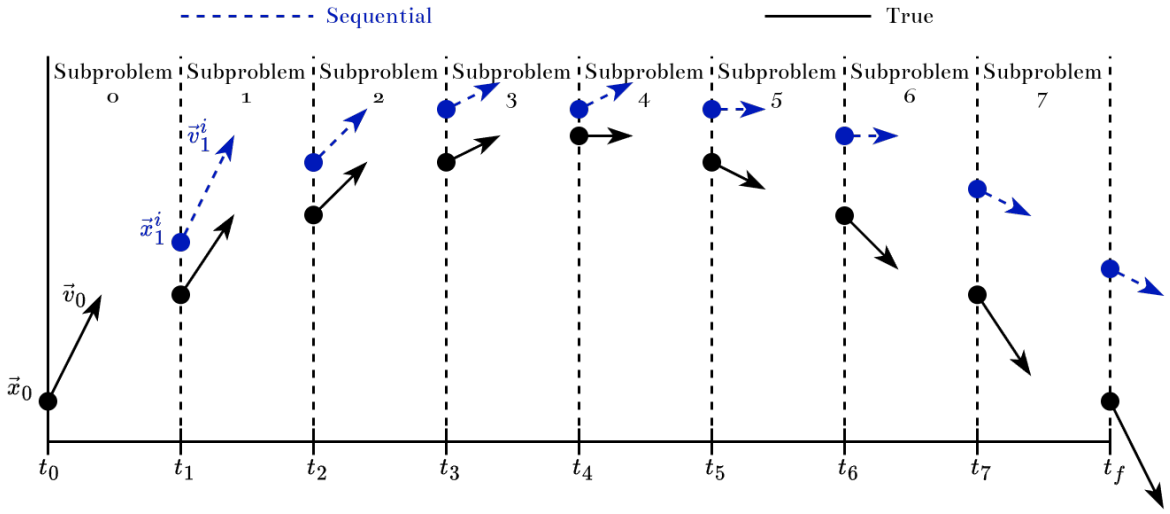

Figure 2: The motion of a ball flying through the air can be partitioned in time to form several initial value problems, each with its own initial position and velocity (upper, blue) determined by a fast integration method. Compared to the true solution (lower, black), this solution is very inaccurate.

3.2.2. Solving the subproblems

Discretizing the domain and propagating initial values as in the previous section constitutes the application of the *coarse propagator* $\mathcal{G}$ to the root problem. Because each of the produced subproblems is independent of the others, each can be accurately solved in parallel using a *fine propagator* $\mathcal{F}$ to reduce the total runtime by a factor equal to the number of subproblems. In other words, if applying $\mathcal{F}$ to a single subproblem has a runtime $\tau_{\mathcal{F}}$, then applying $\mathcal{F}$ to the root problem directly has a runtime $N * \tau_{\mathcal{F}}$ because there are N subproblems, whereas applying $\mathcal{F}$ in parallel only results in a runtime $\tau_{\mathcal{F}}$ because each application of $\mathcal{F}$ executes at the same time.

In general, a **propagator** $\mathcal{P}$ is defined by two key components: its integration algorithm $I_{\mathcal{P}}$, and its discretization $N_{\mathcal{P}}$. More specifically, the propagator $\mathcal{P}$ can be interpreted as a higher-order function mapping integration algorithms and discretizations to functions that, when applied to an IVP, reduce to sequences $\{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t$ of time-position and time-velocity pairs, respectively; the collection of the sequences is called the **solution** S_P of P. The solution can be interpreted as the set of function-graphs (as defined in [34]) of $\vec{r}$ and $\vec{v}$, and is defined such that

$$\mathcal{P}(I_{\mathcal{P}}, N_{\mathcal{P}})(P) = \left\{\{(t, \vec{r}_t)\}_t, \{(t, \vec{v}_t)\}_t\right\} =: S_P. \tag{8}$$

The application of the propagator to the subproblem is the core, or **kernel**, of the PA. While this description of the kernel is useful for understanding, it does not immediately lead to an algorithm that is well-suited for hardware-agnostic implementation (more details in section 4.2 on page 23). To that end, the implementation of the kernel presented here is composed of discretizing the subdomain and propagating the initial values separately.

Discretization: To avoid performance losses from each kernel allocating memory, each discretization kernel references a specific, pre-allocated, one-dimensional array δD of length $N_{\mathcal{P}}$. In order for the kernel to generate the appropriate samplings of the domain, δD has its first and last elements pre-populated with the values of the lower and upper bounds of that thread's assigned subproblem such that

$$\delta D = \left\{t_p, [N_{\mathcal{P}} - 2 \text{ arbitrary elements}], t_{p+1}\right\}. \tag{9}$$

With each kernel accessing this data, it can simply calculate and write the domain samples in-place; this process is shown in Algorithm 2.

Algorithm 2: Parallel Discretization Kernel

INPUT: Discretized domain `ddom` of length `N`
OUTPUT: Nothing

```
a, b ← (ddom[0], ddom[N-1])
step ← (b - a) / 2
for i from 1 to N - 2
    ddom[i] ← a + (i - 1) * step
return
```

Algorithm 2: Each discretized subdomain is calculated by uniformly stepping from the lower bound to the upper bound. These results are written in-place.

Propagation: Much like the discretization kernel, the propagation kernel avoids memory allocations by using pre-allocated, pre-populated multidimensional arrays to store the position and velocity sequences; the initial values of the position and velocity for that kernel's subproblem are pre-populated as their respective arrays first element taking the form

$$\{\vec{r}_t\}_t = \left\{\vec{r}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\right\} \quad \{\vec{v}_t\}_t = \left\{\vec{v}_{t_p}, [N_{\mathcal{P}} - 1 \text{ arbitrary elements}]\right\} \tag{10}$$

Otherwise, the propagation kernel is no more than a traditional IVP solver as described in Section 3.1.2, but executed on many different subproblems simultaneously over many threads. This process is shown algorithmically in Algorithm 3.

Algorithm 3: Parallel Propagation Kernel

INPUT: A solver `solve`,
acceleration function `acc`,
sequence of `N` position vectors `pos_seq`,
sequence of `N` velocity vectors `vel_seq`
OUTPUT: Nothing

```
for i from 2 to N - 1
    old_pos, old_vel ← pos_seq[i - 1], vel_seq[i - 1]
    pos_seq[i], vel_seq[i] ← solve(old_pos, old_vel, acc, step)
return
```

Algorithm 3: Each subproblem is solved in parallel using traditional methods.

The Parareal Kernel: In summary, the Parareal kernel (Algorithm 4) is launched on each thread simultaneously and uses the fine propagator to populate arrays prepared from the domain and initial values of that thread's problem. The domain is discretized by uniformly stepping from the lower bound of the problem's domain to the upper bound. The initial values are propagated using a traditional integration method (Figure 3). The result of this process is sequences of times, positions, and velocities that solve that thread's problem for different discretizations.

Algorithm 4: The Parareal Kernel

INPUT: a fine propagator `fine`, discretized domain `ddom`,
position sequence `pos_seq`, velocity sequence `vel_seq`
OUTPUT: Nothing

```
Use fine in the discretization kernel to populate to ddom
Use fine in the propagation kernel to populate pos_seq and vel_seq
return
```

Algorithm 4: Put it all together

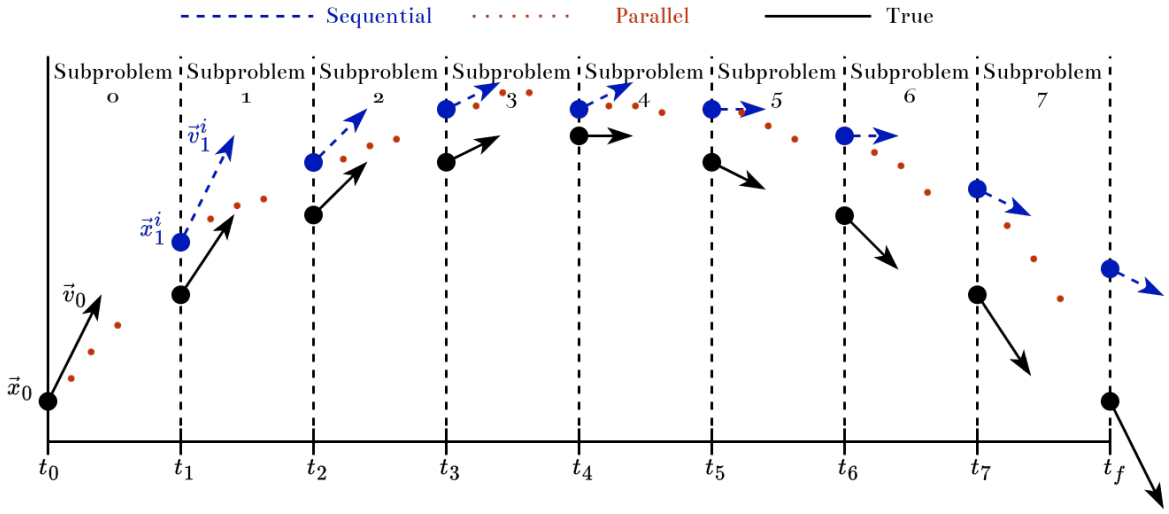

Figure 3: Each thread uses coarse and fine propagators to produce intermediate values (small, red dots) from the initial values (big, blue dots and arrows) of its assigned subproblem. Velocity data does exist, but is neglected here for visual clarity.

Example: For each of the 8 subproblems described by Eq. 7, each taking the form

$$P_p = \left\{\partial_t^2 \vec{r} = \vec{g}, \quad \vec{r}(0) = \vec{r}_p^{\,0}, \ \vec{v}(0) = \vec{v}_p^{\,0}, \quad \left[t_p, t_{p+1}\right]\right\}.$$

1) Define the fine propagator $\mathcal{F}$ as the Velocity-Verlet method with a discretization of $N_{\mathcal{F}} = 2^6 = 64$.
2) Prepare arrays
   2.1) Allocate $3 * 8 = 24$ arrays of length 64 for the fine domains, positions, and velocities.
   2.2) Populate the beginning and end of each domain array with the lower and upper bounds, respectively, of each problem's domain.
   2.3) Populate the beginning of each position array with the initial position of each problem.
   2.4) Populate the beginning of each velocity array with the initial velocity of each problem.
3) Launch the Parareal Kernel with the propagators and prepared arrays
   - **Note:** If there are more problems than threads, the kernel can index stride [35]; more on this in Section 4.2.

3.2.3. Solving the root problem

Because the root solution has been calculated via an inaccurate method, it can be made more accurate using the results of the fine propagator. Though the root solution is not corrected only with the results of the fine propagator, but rather by coarsely propagating the root initial values again but adding a corrector determined by a combination of the results of the fine propagator and the previous iteration's root solution. The main idea of this process is known as *Deferred Corrections* [36].

The correction phase, as defined in literature [2], takes the deceptively-simple recursive form

$$u_t^i := \underbrace{\mathcal{G}(u_{t-1}^i)}_{\text{predictor}} + \underbrace{\mathcal{F}(u_{t-1}^{i-1}) - \mathcal{G}(u_{t-1}^{i-1})}_{\text{corrector}}, \tag{11}$$

where $u = \vec{r}, \vec{v}$ represents either position or velocity of the root problem. If the coarse propagation terms are collected as $\Delta_i \mathcal{G}_t^i := \mathcal{G}_t^i - \mathcal{G}_t^{i-1}$, Eq. 11 can be interpreted as *shooting method in time* [33]. It should also be noted that because the values returned by the coarse propagator are identical in successive iterations i.e. $i \to i+1 \implies \mathcal{G}(u_{t-1}^i) = \mathcal{G}(u_{t-1}^{i-1})$, they can be memo-ized and not calculated again (see Algorithm 5), leading to performance increases at the cost of storage space [37].

One way to interpret the correction equation is "at face value" as

> *On the current iteration, use the coarse propagator to recommend[1] what this value should be. Then correct it by adding the difference between the fine and coarse predictions from the previous iteration. Record this sum as the actual value.*

An alternative interpretation arises from, effectively, "moving the correction to the top of the loop" as

> *Use the coarse propagator to traditionally evolve the root problem, but with an offset. This offset is initially zero.*

This interpretation changes "*use the propagator, then correct the value*" to "*correct the propagator, then use the value*".

Thus the overall behavior of Eq. 11 could be defined through the explicit recurrence relation

$$\begin{aligned} u_t^i &= \mathcal{G}_t^i(u_{t-1}^i) \\ u_0^i &= u(0). \end{aligned} \tag{12}$$

Here the coarse propagator is effectively parameterized and can vary throughout iterations and times such that $\mathcal{G}_t^i(u_{t-1}^i) := \mathcal{G}(u_{t-1}^i) + \Delta_t^i$ and

$$\begin{aligned} \Delta_t^i &= \mathcal{F}(u_t^i) - \mathcal{G}(u_t^i) \\ \Delta_t^0 &= 0. \end{aligned} \tag{13}$$

Eq. 12 has the same structure of a traditional propagation making it much simpler to understand its behavior. This understanding is made even easier when considering Eq. 13 as it allows the Eq. 12 to also define the original coarse propagation of the root problem. In other words, the corrected coarse propagator has a correction of zero on the original iteration. While this interpretation is easier to understand, the result is mechanically identical to Eq. 11.

[1] This terminology is inspired by Giordano & Nakanishi's interpretation of the Gauss-Seidel and Simultaneous Over Relaxation methods in their seminal text on computational physics [38].

Algorithm 5: New Solution Generator

INPUT: Coarse solver `coarse`,
Coarse position array `cpos`, Coarse velocity array `cvel`,
Fine position array `fpos`, Fine velocity array `fvel`,
Root position array `psol[i-1:i, :]`, Root velocity array `vsol[i-1:i, :]`
OUTPUT: Nothing
// evaluated at iteration `i`

```
psol[i, 0], vsol[i, 0] ← p0, v0
for t from 1 to N-1
    ppred[i, t], vpredl[i, t] ← coarse(psol[i, t-1], vsol[i, t-1])
    psol[i, t] ← ppred[i, t] + fpos[i-1, t] - cpos[i-1, t]
    vsol[i, t] ← vpred[i, t] + fvel[i-1, t] - cvel[i-1, t]
```

Algorithm 5: New position and velocity solutions are generated by coarse propagating the previous solution in the current iteration and combining it with the difference between the fine and coarse solutions from the previous iteration. The new solutions are pushed to a the end of the solution arrays.

Example:

1) Have arrays of positions and velocities at each time for the previous and current iteration; the first element of the current iteration's array is the initial value of the problem, while the rest of empty. Also have arrays of positions and velocities generated from the coarse and fine propagators at each time for the previous iteration.
2) Use the new solution generator algorithm with these arrays and the coarse propagator.
3) The arrays for the position and velocity of the root solution at the current iteration are now populated.

3.2.4. Converging the root solution

Finally, as shown in Algorithm 6, launch the Parareal kernel (Algorithm 4) to gather the fine solutions for each point in time, and construct the new root solution (Algorithm 5) until the root solution changes less than a ceratin amount, by some metric, at which point it has converged. While there are many choices that can serve as valid convergence criteria [33], one of the simplest is:

$$\max_{1 \leq t \leq N-1} |u_t^i - u_t^{i-1}| < \varepsilon, \tag{14}$$

for some threshold ε. Eq. 14 determines convergence when every point in the solution changes by less than some amount between iterations.

Algorithm 6: The Parareal Algorithm

INPUT: Root problem `P`, Coarse propagator `G`, Fine propagator `F`, Convergence threshold `ep`
OUTPUT: Discretized domain `ddom`, Root position sequence `pos`, Root velocity sequence `vel`

```
1 ddom, pos[0, :], vel[0, :] ← Prepare the subproblems via a coarse solution
2 i ← 1
3 while max(changes) ≥ ep
4     fpos[t], fvel[t] ← Launch the parareal kernel in parallel with F, pos[i-1, :], vel[i-1, :]
      to get the fine solutions for subproblem t
5     pos[i, :], vel[i, :] ← Construct new root solutions with G, pos[i-1, :], vel[i-1, :],
      and fpos, fvel
6     changes ← The difference between the current and previous solutions
7 return ddom, pos, vel
```

Algorithm 6: The PA is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

The magic of the PA lies in its divide-and-conquer approach to solving initial value problems. The "root" problem is sequentially and inaccurately solved to divide it into smaller problems whose initial values are defined by the solution. Those problems are simultaneously and accurately solved in parallel. The root problem is then solved in the same way as before, but at each step, the data is modified by combining the previous accurate and inaccurate solutions. Finally, the new root solution defines new problems, and the loop continues until the the solution has converged.

4. The Parareal Algorithm at Scale

Section 3.2 highlights the PA's nature of being quasi-embarrassingly-parallel i.e. the performance of the algorithm scales with the number of threads, while still being bottlenecked by a periodic sequential process. That being said, the previous discussion ignores the details and nuances of implementing the PA including the actual form of the threads. The primary goal of this work is to provide two new implementation models that both take advantage of increased parallelism and allow easy scaling: using the massively parallel architecture of graphics processing units (GPUs), and the scalability of distributed systems. Instances of these implementations are also provided [39].

The PA as defined in Algorithm 6 does not change in it *what* it does when implemented to use GPUs, but rather *how* it does. There are several issues that arise when utilizing general-purpose GPU computing (GPGPU) such as the GPU needing to wait until the CPU tells it to do something, better performance with less precision, and the restriction to using primitive types like "ints" and "floats". Though, the biggest issue is the need to consider the movement of data between RAM and VRAM, or more generally host memory and device memory; considering unshared memory spaces will be even more important in section Section 4.3.

The distributed-based implementation focuses on distributing problems across multiple remote machines. These machines solve their problems simultaneously with the other machines, thus achieving a form of parallelism only limited by the number of accessible machines. These problems could either arise from the coarse propagation of a single root problem, yielding a "problem tree" () where each machine would create-distribute-collect its own set of problems, or if there are multiple "true root" problems e.g. a system of ODEs.

The GPU- and distribution-based methods can be combined to further parallelize solving an initial value problem. If each of the machines available to the distributed network has at least one GPU (a single machine can have multiple; more details in Section 4.3), this implementation will automatically identify, manage, and use all of them. Thus these methods can be composed to provide a scalable model of parallel-in-time integration for equations of motion.

High-performance computing (HPC) (section Section 4.1), in the context of this work, focuses on utilizing two core ideas: **multithreading & GPU computing**, and **multiprocessing & distributed computing**. These ideas contrast *sequential* procedures where the next calculation cannot be started before the previous has finished. Multithreading, and more specifically using graphics processing units (GPUs) to do general purpose computation i.e. GPGPU computing, allow several calculations to be done simultaneously on the same physical hardware i.e. in *parallel*. Further extending this idea, multiprocessing (not to be confused with multi-*threading*) allows calculations to be executed simultaneously as in the case of several threads, but these calculations "have their own set of knowledge". This seemingly subtle distinction provides the ability for these multiple processes to be executed on *different* physical hardware. These models of parallelism have been used in the past to address the runtime issues arising from simulating complex physical phenomena.

4.1. Review of High-Performance Computing

When problems get big enough, or more concretely when there is enough data that needs to be processed, traditional computers are unable to complete the task in a reasonable amount of time. When such cases arise, not only are more performant computers needed, but also the methods used to *implement* the calculations need to be changed as well. In essence, **high-performance computing** (HPC) is about performing as many as calculations as possible in the least amount of time. One of the most direct methods to reduce the time needed to perform calculations is to "simply" perform multiple of them at the same time, otherwise known as **parallel processing**. There are several ways in which parallel processing can be achieved; some of which being using a multi-core CPU, a GPU, and multiple physical computers.

Section 4.1.1 covers the basics of utilizing multiple *threads* to perform computations simultaneously. In the case of *multithreading* for HPC, even using multiple threads on a CPU is insufficient as CPUs are only capable of performing, at the time of writing, on the order of 10s of calculations simultaneously. For this reason, the massively-parallel architecture of GPUs has been utilized to performing 10s of *thousands* of calculations simultaneously. Though the extra power does not come freely.

Section 4.1.2 covers the basics of utilizing multiple *processes* and even multiple physical machines to perform *many* calculations simultaneously. Much like multithreading, the potential performance increase from utilizing multiple processes on a single CPU is still limited by its "10s of calculations" ability, and in fact is lower than with multithreading due to processes needing more resources to exist. Unlike multithreading, multiprocessing allows for calculations to be distributed amongst resources that are not even physically located on the same machine, allowing for theoretically *unlimited* performance increases. Though, like multithreading, this utilizing this power does not come without its challenges.

One of the core goals of this work is to combine the power of massive multithreading from GPUs and the unbound potential from distributed computing to solve EOMs. This will allow "extreme-scale", time-dependent problems to be solved in reasonable time.

4.1.1. Multithreading & GPU Computing

For the purposes of this work, there are three ideas that need to be understood. The first is that data needs to be copied between the CPU and the GPU. The second is how the GPU can read and write that data safely. Finally, the third is how the GPU performs calculations on that data. A note on terminology: when discussing GPGPU computing, the nomenclature refers to the memory and overall system accessible to the CPU as the **host**, and similarly the memory and resources available to the GPU is known as the **device**.

When data is processed and stored by the host in RAM, that data is not automatically accessible to the device. The device has its own memory and can only read and write to it, so any data that is used on the GPU must first be copied to it. When the device has finished its calculations and written the new data to its memory, that data must then be copied from the device to the host.

This host-device communication is orders-of-magnitude slower than the communication between the resources on the device [40], and thus should be minimized. Note this does not take into account the time needed to *allocate* memory, which only exacerbates the issue.

The data that's transferred between the host and device usually takes the form of an array. Because each thread on the device executes the same program (called the **kernel**), each thread needs to access different elements of the array based on its position relative to the other threads. This follows the *single instruction, multiple thread* (SIMT) model of parallelism. This pattern of indexing is shown in Figure 4.

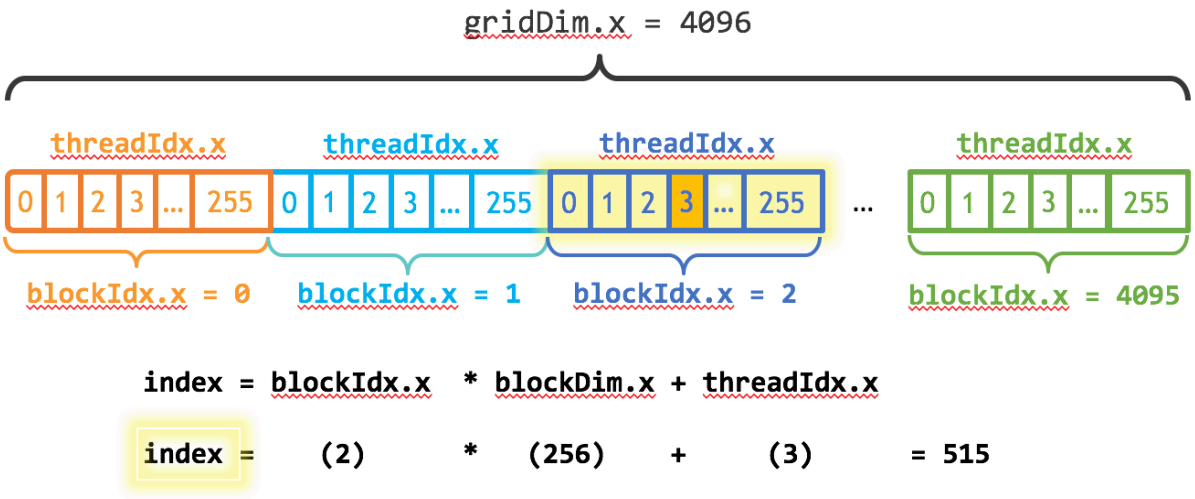

Figure 4: Each thread accesses an index of the array (`index`) based on the thread's location (`threadIdx.x`) in its block, how many threads there are in its block (`blockDim.x`), and the block's location (`blockIdx.x`) in the grid. Image credit [41].

When there are more elements in the array than there are threads on the device, each thread must process multiple array elements. Once each thread is finished writing to its index (either in-place to the original array, or to another array copied from the host), it "jumps over" all the indices that were just written to by all the other threads in all the other blocks, and writes to the next one. The number of indices the thread "jumps", called the *stride*, is determined by the number of threads in each block (`blockDim.x`) and the number of blocks in each grid (`gridDim.x`). This is known as *index striding* and is frequently used in GPU programming to process arrays of arbitrary dimension [35]. This idea is shown in Figure 5.

Once each thread knows its array index, all threads execute the kernel simultaneously. This is where the increased performance comes in. If the program takes T time to execute on a single array element, and there are N array elements, then the total time to calculate sequentially would be TN. Because the device processes each element simultaneously (as long as there are more threads than elements), the total time to calculate is that of a single execution i.e T. If there are M times as many elements are there are threads, then the total time would simply be MT as each thread

processes M elements. Further parallelization can be achieved by distributing the array elements over multiple devices and machines.

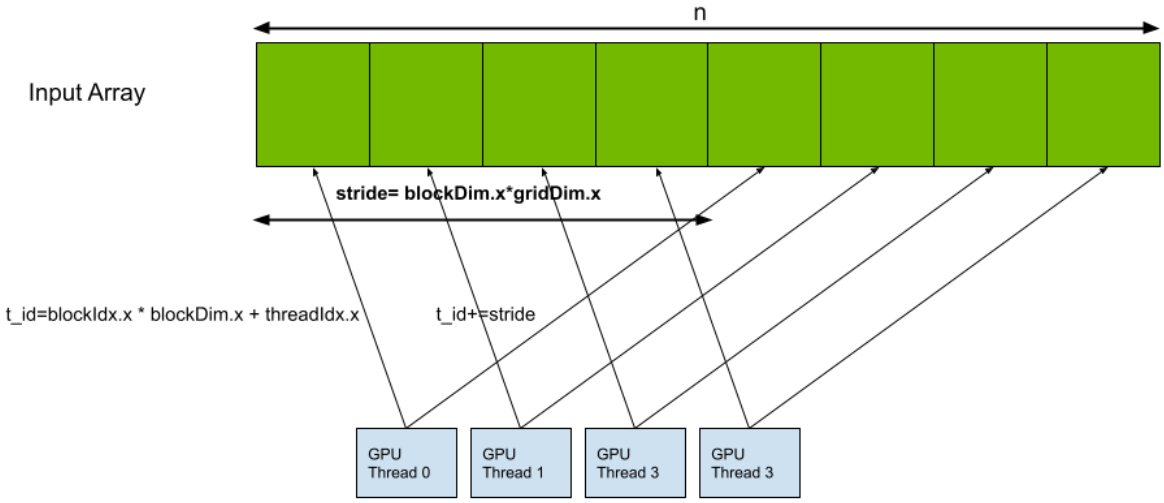

Figure 5: The indices a specific thread processes are based on how many total threads there are. Image credit [42]

4.1.2. Multiprocessing & Distributed Computing

The three most important ideas to understand in multiprocessing and distributed computing for this work are: different processes do not have access to the same data, one process can tell another to perform an action, and the time associated with communicating between processes. To distinguish processes and threads, consider two homes A and B each with its own family. Each home represents a process with its associated family members being that process's threads.

Unlike threads, the context, or *memory space*, a process has is distinct from that of other processes. Thus, if something changes in one process, other processes are unaware of the change. Considering the analogy, if home A's phone is moved from the dining room to the living room by Alice who then writes the new location on the fridge, the other family members in home A can all see this change because they all have access to the same fridge. As for home B, not only does the home phone not move, but family B also does not know home A's phone moved, because they do not have access to home B's fridge. It is only when information is explicitly communicated between these processes/homes that the other has this new information.

The communication of data between between processes is much slower and requires much more overhead than communicating between threads. Considering the analogy, when a family member in home A can't find the phone, they simply go to the fridge for the updated information. If, for some reason, family B needs the location of family A's phone, someone from family A would probably walk over to home B and update them. This takes much more time and resources than going to the fridge in the same home (and even more time and effort is required to update in the next town over!). But what if Bob in home B wants Alice in home A to do something?

Remote Procedure Calls: While there are many methods to communicate information between processes, the most important method for this work involves one processes telling another process to execute some procedure, which is aptly named a *remote procedure call* (RPC). RPC allows one process, such as the one launched by a user running a program, to direct another process to execute some command using its own resources. Though, because each process has its own memory space, any references to objects that don't exist in the *remote* process e.g. variable, libraries, etc., will fail, and references to objects "with the same name" can produce unintended results.

In the analogy, Alice in home A wants to compare the location of her phone to the location of Bob's phone in home B, so she asks Bob "Where is the phone?". Because both homes have phones, but they are in different locations, Alice would answer "the living room" and Bob would answer "the kitchen" because "the phone" is relative to each home. If Alice and Bob wanted to have the same answer, then either they would have to communicate where they want the phone to be and put it there, or refer to the same physical instance of a phone.

Out of the analogy, each remote host effectively needs to be an identical copy of the local host. This can be achieved by each host referring to a shared file system so they all manifestly the same binaries, versions of packages, etc. This needs to happen because an RPC can be thought of as sending a chunk of raw, textual source code to be run on the other process and/or machine. If that source code calls some functionality that is not loaded or otherwise available in that process, that call will error. Thus things like a GPU library must be not only available on each machine, but also loaded on each process.

4.2. Parareal on the GPU

The GPU-based implementation focuses on three ideas. The first is utilizing the massively-parallel architecture of a GPU to *simultaneously* use orders-of-magnitude more threads than what would be possible with a CPU. The second is considering the movement of data between the host memory (RAM) and the device memory (VRAM). And the last is needing to use primitive data-types. Otherwise, the underlying algorithm is no different than what is presented in Section 3.2.

The PA (Algorithm 6) is only limited by the number of threads at its disposal. When the number of threads is greater than the number of cores, the processor needs to switch thread contexts in order to balance the evolution of each thread. On a CPU, this context switching is very costly and can lead to drastic decreases in performance [43], [44]. On a GPU however, switching thread-contexts is nearly free [45], allowing there to be *many* more threads than processors without sacrificing efficiency. So, it is very beneficial to execute the PA on hardware that not only can efficiently handle many threads, but also have them running at the same time.

All relevant data is first allocated and pre-populated by the CPU on the host. Then the CPU copies that data to the device. Then the CPU tells the GPU to execute the Parareal kernel on its copy of the data, producing solution data. The CPU then copies the solution data from the device to the host and recreates the problems. The transfer of data (and the CPU launching the kernel on the GPU) between the host and the device serves as the main performance bottleneck in this process. [45] Dynamic parallelism can be used to launch kernels directly from the GPU, thus circumventing the performance drawbacks of host-device communication [45].

GPUs can only process primitive types of data such as integers, floats, booleans, and other "bits-types". This precludes collecting the problem and solution data in more intuitive forms like one would do when representing them mathematically. In other words, whereas a CPU is happy to handle several boxes each with its own set of elements e.g. domain, acceleration, initial position, and initial velocity, GPUs need this same underlying data to be collected such that all domains are in one box, all acceleration functions are in another box, all initial positions are another, and initial velocities in another. These "boxes" take the form of arrays. It is for this reason, the PA as described in Section 3.2 uses its data as arrays.

So, why is the PA well-suited to be implemented to use GPUs? Because the data is only composed of numbers, it can be simply represented in a GPU-friendly array structure. The massive number of cores on a GPU can simultaneously process these arrays with a much lower cost of switching between threads and problems. And finally, the solution data can be easily copied back to the host. This process is shown in Algorithm 7 and Figure 6.

Algorithm 7: The GPU-Based Parareal Algorithm

INPUT: Root problem `P`, Coarse propagator `G`, Fine propagator `F`, Convergence threshold `ep`
OUTPUT: Discretized domain `ddom`, Root position sequence `pos`, Root velocity sequence `vel`

```
1  i ← 0
2  ddom, pos[i, :], vel[i, :] ← On the host, prepare the subproblems via a coarse solution
3  while max(changes) ≥ ep
4      i += 1
       // fine propagate on the device
5      posd[i-1, :], veld[i-1, :] ← Copy F, pos[i-1, :], vel[i-1, :] to the device
6      fposd[t], fveld[t] ← Launch the parareal kernel to get the fine solutions for subproblem t
7      fpos, fvel ← Copy the fine solutions fposd, fveld to the host
       // coarse propagate on the host
8      pos[i, :], vel[i, :] ← Construct new root solutions with G, pos[i-1, :], vel[i-1, :],
       fpos, fvel
9      changes ← The difference between the current and previous solutions
10 return ddom, pos, vel
```

Algorithm 7: The PA is composed of looping two steps: finding the fine solutions in parallel, then finding the root solution sequentially. The loop stops when the root solution stops changing.

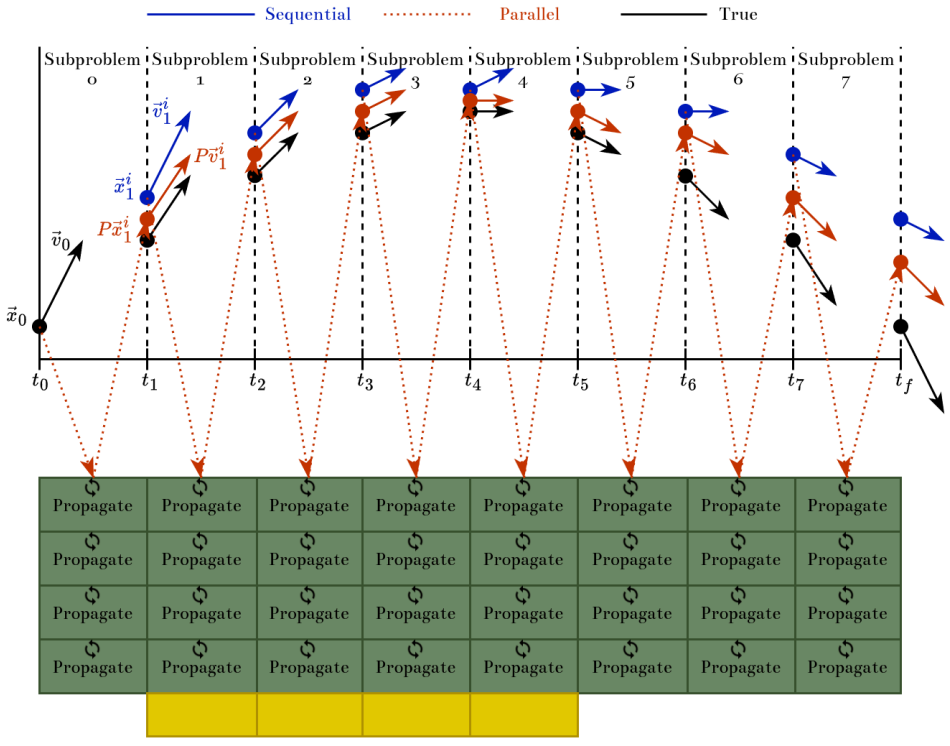

Figure 6: Sequential solutions (top in blue) are sent to the GPU to be finely-propagated (mid in red) in parallel; true solutions (bottom in black) are shown for comparison.

4.3. Parareal on the Cluster

While the PA can be further parallelized using GPUs, the fact still stands that the PA is quasi-embarrassingly-parallel. In other words, each subproblem is independent of the others while each is being solved, and each of these subproblems can be assigned its own thread. So, if there are more threads available, higher performance or accuracy can be achieved. The implementation presented here provides more threads by sending problems to remote machines where they can be run simultaneously; in other words, multiple machines with their own CPUs and GPUs are networked together to form a *cluster* where the work is distributed amongst all machines.

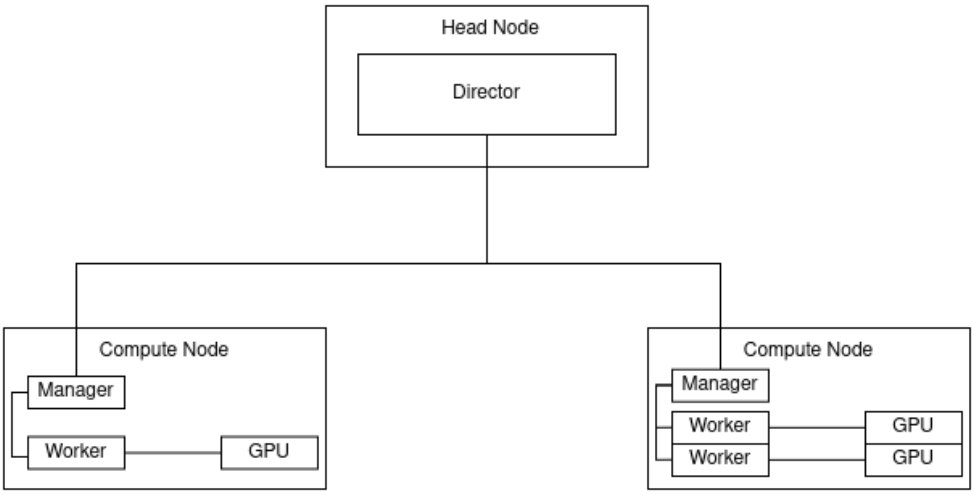

Figure 7: The assumed topology of the cluster presented in this work.

While clusters can take many forms, this implementation considers building a cluster from the ground up in a modular and ad-hoc manner. This way the cluster can theoretically scale without limit. The general structure, or *topology*, of the cluster is rather simple: A *head node* runs the *director* process, which tells *worker* processes on other *compute nodes* what to do; this structure is shown in Figure 7. In other words, the director only decides and does not do, while the workers do not decide and only do. While the workers can solve their problems and communicate with every other worker (and the director), only the director can spawn new processes. Once the director has finished spawning and configuring the workers as in Algorithm 8, the director moves on to begin the PA.

Algorithm 8: Prepare the Cluster

INPUT: A collection of hosts available in the cluster

OUTPUT: A collection of hosts ready to compute

1 Director process spawns manager processes on each host

2 Each manager sends the number of devices on its host back to the director

3 Director process spawns worker processes on each host, 1 for each device on that host

4 Pair each worker with a device on its host

Algorithm 8: The cluster can be created and prepared in 4 simple steps.

In order for this implementation to be flexible, the director does not assume any a-priori configuration of any worker nodes. This way any node can be seamlessly introduced to the cluster. The drawback of this flexibility is that the cluster must be created each time. Part of this creation is identifying the available devices in the cluster. To do this, the director first uses a user-given collection of hostnames to spawn a single "manager" process on each of the given hosts.

Algorithm 9: Spawn Manager Processes

INPUT: A list of hosts

OUTPUT: A list of manager process IDs

1 The director spawns manager process on each host

2 The director tells each manager to load the Parareal library

Algorithm 9: Manager processes are created to identify the number of devices on a host, and can manage the network communication between hosts.

Once the managers have been spawned, the director asks them how many devices are on their host. The manager measures this and sends back the information to the director. The director, because it is the only process that can spawn workers, spawns a number of workers on each host equal to the number of devices on it. This process is shown in Algorithm 10.

Algorithm 10: Spawn Worker Processes

INPUT: A list of manager IDs

OUTPUT: A list of worker IDs

```
The director tells each manager to load the GPU library // provides ability to count devices
The director requests the number of devices on the host from each manager
For each host
    For each device on the host
        The director spawns a process on the host
The director tells each manager to load the Parareal library
```

Algorithm 10: Worker processes are spawned by the director on each host based on the number of devices available to that host.

Once the workers are spawned on their respective hosts, each of them needs a device. While the fundamental idea of assigning a device to a worker is trivial (as shown in Algorithm 11), the implementation suffers from the fact the a device should not be assigned to a process on a different host! In this implementation, process IDs correlate to the order in which they were spawned e.g. process 2 was spawned second, process 3 was spawned third; in other words, the process IDs are relative to the whole cluster. The device IDs, however, are relative to their host machine. So, care must be taken in order to pair processes and devices on the same host.

Algorithm 11: Assign Devices - High Level

INPUT:

OUTPUT:

```
The director tells each new worker to load the GPU library // provides ability to assign devices
For each host
    For each worker on that host
        The worker assigns an on-host device to itself
```

Algorithm 11: Pairing workers and devices is trivial at a high level

Using a cluster management system to keep track of process IDs, thus without loss of generality, consider the following:

- The director has process ID (PID) 1 and is on its own host.
- Host X has PID 2, and host Y has PIDS 3, and 4.
- Host X has 1 device with ID 0, and host Y has 2 devices with ID 0 and 1.
- The desired result is
  - Device 0 on host X is assigned to PID 2
  - Device 0 on host Y is assigned to PID 3
  - Device 1 on host Y is assigned to PID 4

One way to address this issue is to create "host objects" by collecting the hostnames, PIDS, and number of devices for each host. The collection of these host objects, together with their relative connections, would constitute a "cluster object" or "cluster graph". While the actual ids of the devices on each host are what is important, the pattern is the same for all hosts e.g. `devids = 0, 1, ..., ndevs-1`. This way only a single integer needs to be stored instead of a list.

Algorithm 12: CREATE HOST OBJECTS

INPUT: List of hostnames, list of managers, list of device counts
OUTPUT: List of host objects `hosts`

```
For each host
    name ← name of host
    workers ← list of woker IDs on host
    ndevs ← number of devices on host
    hosts[i] ← Host(name, workers, ndevs)
return hosts
```

Algorithm 12: To aid in the pairing of devices and workers, host objects can be created to make sure devices are assigned to workers on the same host.

A host object packages together the worker IDs and number of devices on the same host. Thus devices can be assigned to workers on the same machine simply by iterating through the host objects. This process is shown in Algorithm 13.

Algorithm 13: ASSIGN DEVICES

INPUT: List of host objects `hosts`
OUTPUT: Nothing

```
Load the GPU library on each worker // provides ability to assign devices
for host in hosts
    workers ← host.workers
    devids ← (0, 1, ..., host.ndevs-1)
    for each pair (worker, devid)
        Assign devid to worker
```

Algorithm 13: A more detailed algorithm of assigning devices to workers on the same host.

Once the devices are assigned, the cluster has been prepared. The director then becomes the driver of the PA, as shown in Algorithm 14. The director begins the PA as it normally would, but instead of either dispatching the parallel propagation to other CPU threads or the GPU, the director gives each worker a problem to propagate in parallel, either on its CPU or GPU. Since this dispatching takes little time, the director then waits until it gets the solutions from the the workers. Finally,

the director coarse propagates the root problem with the solutions, checks if the solution has converged, and loops if it hasn't.

Algorithm 14: The Parareal Algorithm at Scale

INPUT: A root problem, coarse and fine propagators, and a convergence threshold
OUTPUT: A solution to the root problem
// on a prepared cluster

```
1 The director coarse propagates root problem to get initial solution
2 While the root solution has not converged
3     The director creates problems
4     For each problem
5         The director sends the problem to a worker // only repeating once all workers have been assigned at least one problem
6         The director tells the worker to solve the problem using the Parareal algorithm on its GPU
7         The director requests the solution from the worker
8     The director waits to get all solutions
9     The director coarse propagates root problem with corrections to get new solution
```

Algorithm 14: The PA can distribute its subproblems amongst multiple machines in order to achieve higher performance.

5. Performance Analysis

While there are many significant aspects of this work that could be analyzed, only three that will be considered here. The first is on the quality of the results produced by this implementation via standard numerical analysis. The effects of transferring data between the host and the device repeatedly arises only in the implementation and the theoretical nature of this bottleneck is investigated. Similarly, the significance of transferring data between hosts is analyzed. And finally, empirical benchmarks are given to highlight the efficacy of the implementation.

Section 5.1 focusses on measuring the effects of numerical approximation. Some of the most notable effects to consider are error, stability, and convergence. Instead of considering raw error, in this work energy-drift will be used as a proxy (as outlined in Section 3.1.2). In this case, stability refers to how the error of the result changes with respect the length of the time domain of the root problem. Convergence measures how many iterations the implementation takes to converge against the coarse and fine discretizations.

Section 5.2 focuses on identifying sources of latency due to data transfers. More specifically, those transfers that occur between the host memory and the memory on the GPU. Strategies to mitigate or completely circumvent these latencies are suggested.

Section 5.3 focuses on comparing the performance of different forms of parallelism. To provide a baseline measurement, the performance of a single threaded integration is provided across ranges of coarse and fine discretization. Similarly, the performances of a local GPU and the distributed implementation provided by this work are measured for varying discretizations. Finally, the performances of each of these methods are compared to highlight which method has the best and worst runtime.

5.1. Numerical Analysis

Because the PA acts as a "meta-algorithm", the underlying integration methods must also be chosen. For these results, the integration schemes for the coarse and fine propagators are the symplectic-Euler and velocity-Verlet methods, respectively. The symplectic-Euler method is chosen for the coarse propagation because it computationally "cheap" while still being symplectic. The velocity-Verlet method is chosen for the fine propagation due to its higher accuracy, while still being computationally inexpensive. While these methods are similar in their computational cost and accuracy for a single propagation, the multiple resolutions of the time domain provide the ability for the fine propagator to have a higher discretization and thus a much smaller time-step compared to the coarse propagator.

An important item to note is that data here is represented using only 32 bits instead of the de-facto standard of 64. This restriction is because GPU performance is significantly better when using 32-bit floating point representations ("32-bit floats") compared to using 64. Though, when both the coarse discretization and the integration method are inaccurate (e.g. $N_{\mathcal{G}} = 4$ with the Euler method), the solution diverges to yield data that is too large to be represented with only 32-bits (the maximum being $\approx 10^{38}$). Additionally, using only 32 bits increases round-off error and thus accelerates the solution's divergence. Regardless of the source, these overflows result in a $\pm\infty$.

5.1.1. Discretization Error & Energy Drift

The error associated with a simulation depends on many factors, but one of the most controllable is that associated with the time-step: the smaller the time-step the closer the simulation is to reality. The time step used by the coarse propagation only depends on the coarse discretization as $\Delta t_{\mathcal{G}} = T/N_{\mathcal{G}}$ while the time step used in the fine propagation depends on both the coarse and fine discretizations as $\Delta t_{\mathcal{F}} = \Delta t_{\mathcal{G}}/N_{\mathcal{F}} = T/N_{\mathcal{G}}N_{\mathcal{F}}$. This section analyzes how the accuracy of the simulation depends on the size of each of these time steps.

Additionally, in order to understand the error of a numerical approximation, it must be compared to a known value. For this analysis, that reference is the constant energy E_{true} of a simple pendulum under no dissipative nor driving forces. The error is defined by the deviation of the simulated energy E_{sim} from the true energy after the pendulum has undergone ten oscillations. In order to ensure measurements are scale independent, the difference in energy is scaled by the true energy to yield the relative energy drift $E_r = (E_{\text{sim}} - E_{\text{true}})/E_{\text{true}}$. In this case the has unit mass $m = 1$ kg pendulum begins at its low point $\theta_0 = 0$ rads a distance $\ell = 1$m below its pivot with unit velocity $\omega_0 = 1$ rads/s yielding $E_{\text{true}} = m\ell^2\omega_0^2/2 = 0.5$J.

Figure 8 shows how the error depends on the coarse discretization for several choices of fine discretization. Notably, for every presented fine discretization, the error seemingly depends not just nonlinearly, but non-monotonically. Yielding almost quadratic behavior, the error initially decreases for an increase of coarse discretization, reaching a minimum at a coarse discretization of 2^{10} for every fine discretization, then rising again. This nonintuitive behavior warrants further

investigation as the error is expected to exponentially decay converging to zero for larger coarse discretizations.

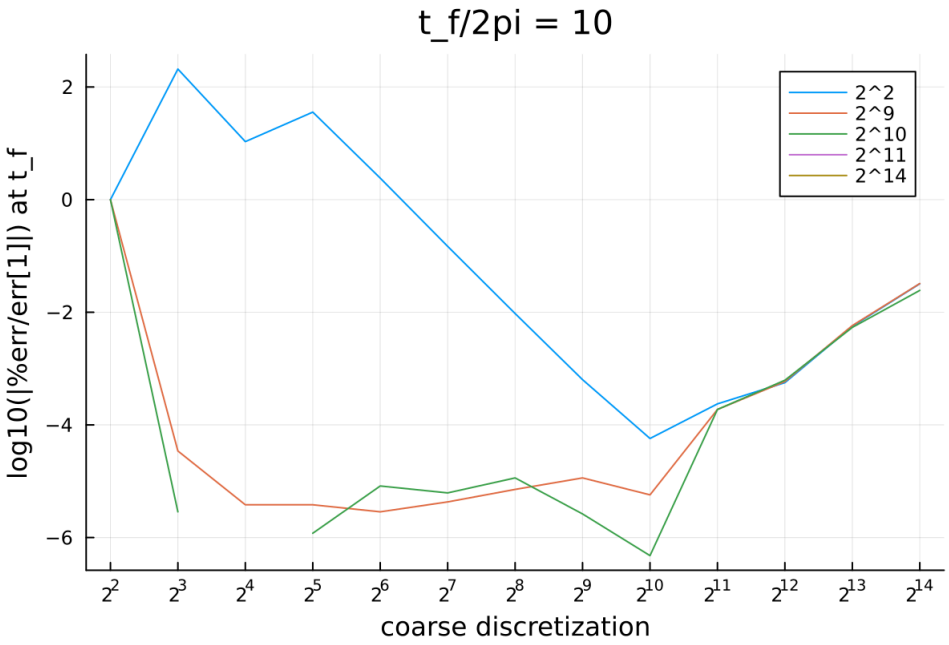

Figure 8: The order of magnitude of the normalized percent error of the final state of the pendulum for different coarse and fine discretizations.

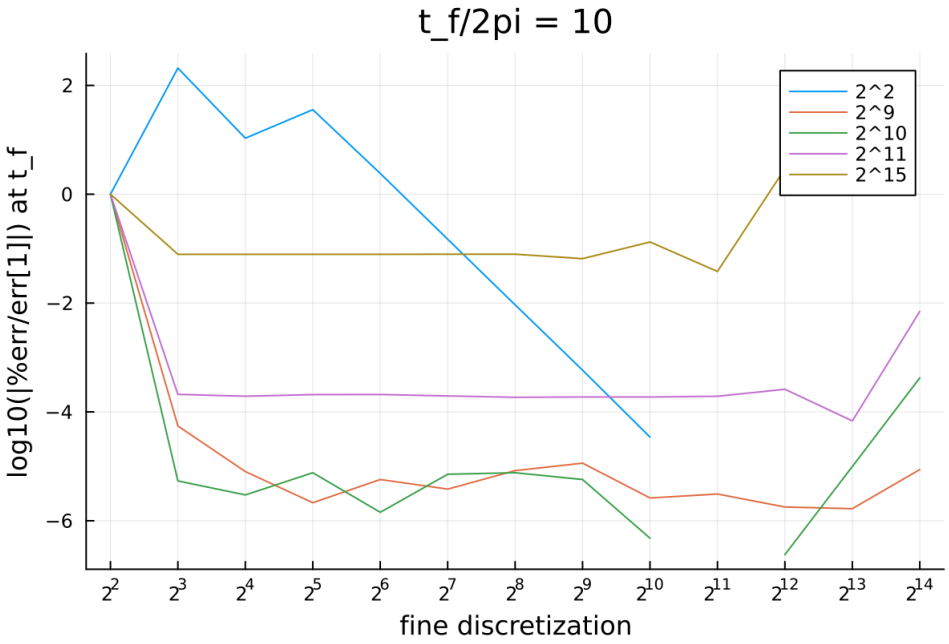

Figure 9: The order of magnitude of the normalized percent error of the final state of the pendulum for different coarse and fine discretizations.

Figure 9 shows how the energy drift of the simulation is affected by the fine discretization for a particular choice of the coarse discretization. There are two key features that should be noted here: the error decreases significantly even for a small change in the fine discretization but then plateaus only to increase at large values of the fine discretization, and the difference in the difference of error (i.e. $\Delta^2 E_r / \Delta N_{\mathcal{F}} \Delta N_{\mathcal{G}}$) is non-monotonic for different fine discretizations. The former signifies that the quality of the results produced from this implementation does not significantly depend on the fine discretization, until it becomes large. The latter suggests there is a complex topography of the error-discretization space that warrants further investigation; a possible starting point would be to explore the fact that $E_r \propto \Delta t \propto 1/\Delta N_{\mathcal{G}} \Delta N_{\mathcal{F}}$.

The results shown in figures 8 and 9 show that the error of using this implementation does depend on the size of the coarse and fine discretizations. More specifically, the error seems to be concave in each of the discretizations, first decreasing with larger discretization before reaching a minimum and then increasing. Furthermore, there also seems to be concavity in the mixed change of the error $\Delta^2 E_r / \Delta N_{\mathcal{G}} \Delta N_{\mathcal{F}}$. These conclusions warrant further investigation of the topography of the error-discretization space of this implementation and of the PA itself.

5.1.2. Stability

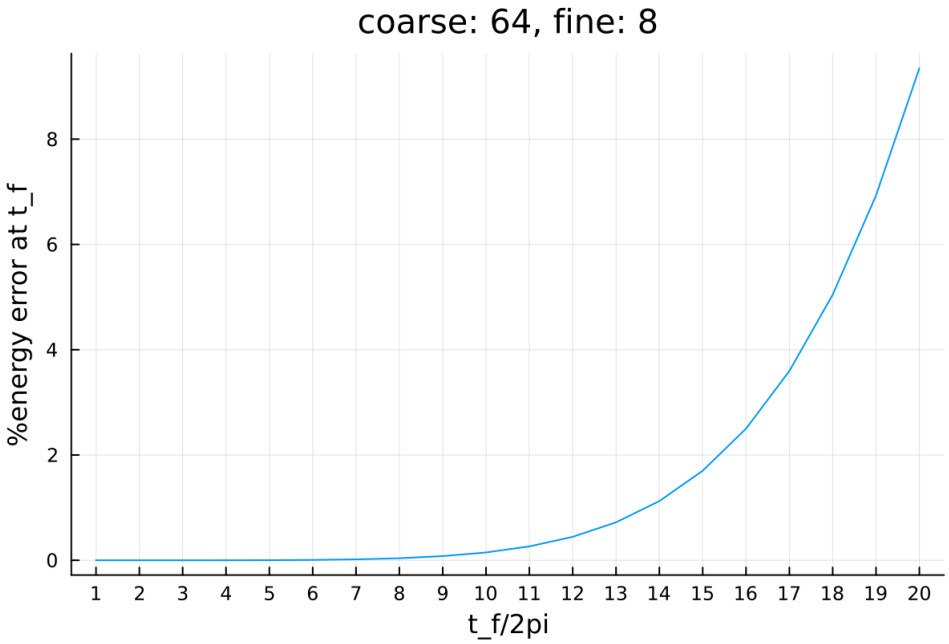

Figure 10: The global error of the simulation quadratically increases as the length of the simulation/ the number of iterations increases. This is consistent with the analytically determined global error of the velocity-Verlet algorithm being $O(\Delta t^2)$.

The notion of stability in the context numerically solving differential equations can refer to the tendency of an integration algorithm to "blow up" due to the accumulation of error. As each iteration of the algorithm introduces error, the stability of a simulation depends on the number of iterations it undergoes. For integrating equations of motion, a simulation can iterate more times N for two variations of $t_f - t_i = N\Delta t$: the final time t_f of the simulation becomes larger while the time step Δt stays the same, or the time step Δt becomes smaller while the time domain $t_f - t_i$ does not change. While the consequences of the former are relatively simple as the error introduced with each iteration accumulates more and more to create the global error, the latter involves both the decrease in error associated with decrease in time-step and the increase in error associated with the increase of the number of iterations.

Figure 10 shows the global error of the simulation as the time-step remains constant and the final time, and thus the number of iterations, increases. The results of the PA are identical to those that would be produced by the using the fine propagator by itself, which in this case is the velocity-Verlet method. The global error shown is consistent with the known behavior of the global error of the velocity-Verlet algorithm $O(\Delta t^2)$, which is used here.

5.1.3. Convergence

How quickly a sequence of approximate solutions approaches the true solution (usually asymptotically) is known as its rate of convergence (RoC), and applies to both iterative and discretizing algorithms. For iterative algorithms, such Newton's Method and Gauss-Seidel Relaxation, the RoC is determined by how many iterations the algorithm needs to undergo for successive solutions to change within some threshold. For discretizing algorithms, such as the Finite-Difference and Finite-Element Methods, the RoC is determined by how much the solution changes for increasingly accurate approximations of the domain. Because the PA is both iterative and discretizing, these rates depend on each-other and thus the number of iterations needed to converge depends on the discretizations of the domain. It should also be noted that the PA converges in at most a number of iterations equal to the coarse discretization [33].

Figure 11 shows how the number of iterations the simulation needs to converge depends on the coarse discretization. For a coarse discretization less than $2^6 = 64$, the RoC is nearly linear, but for a coarse discretization of $2^7 = 128$ the RoC is much less than the coarse discretization e.g. a coarse discretization of $2^{16} = 65,536$ converges in approximately 400 iterations; the latter behavior is known as superlinear convergence [46]. Both the linear convergence for small coarse discretizations, and the superlinear convergence for greater coarse discretizations are consistent with literature [33], [47].

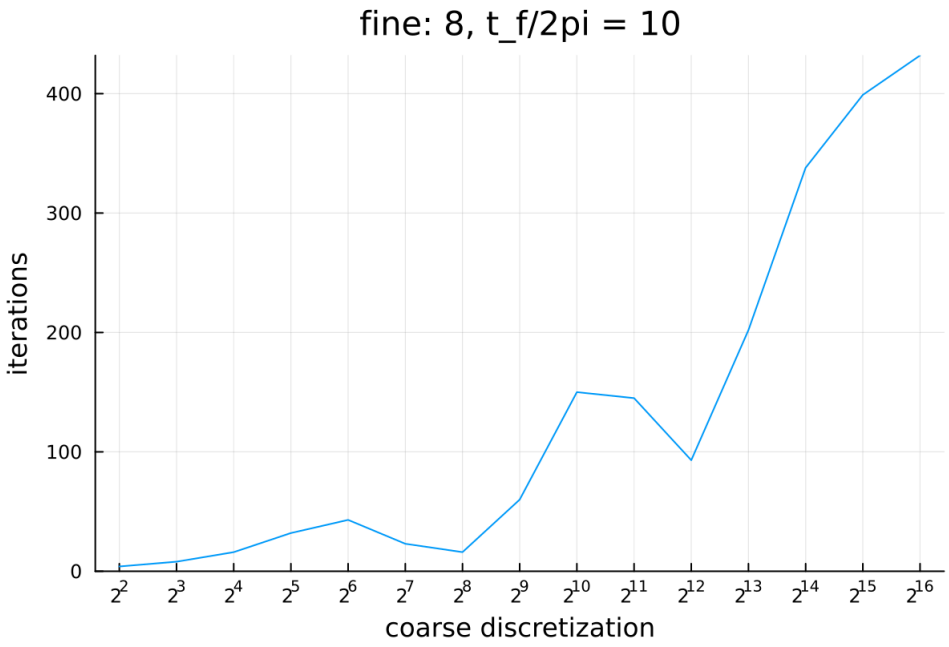

Figure 11: The number of iterations the simulations needs to converge to a solution versus the coarse discretization of the domain. These results show linear and superlinear rates of convergence for small and large discretizations, respectively.

Figure 12 shows how the RoC depends on the fine discretization. Compared to how the RoC depends on the coarse discretization, that for the fine discretization is rather simple as it is nearly the same for all fine discretizations. Though while simple, the results seem to be contrary to the expectation that a larger fine discretization leads to more accuracy and thus fewer iterations. Unfortunately, the details of how the RoC depends on the fine discretization and solver are nuanced and out of the scope of this analysis [33].

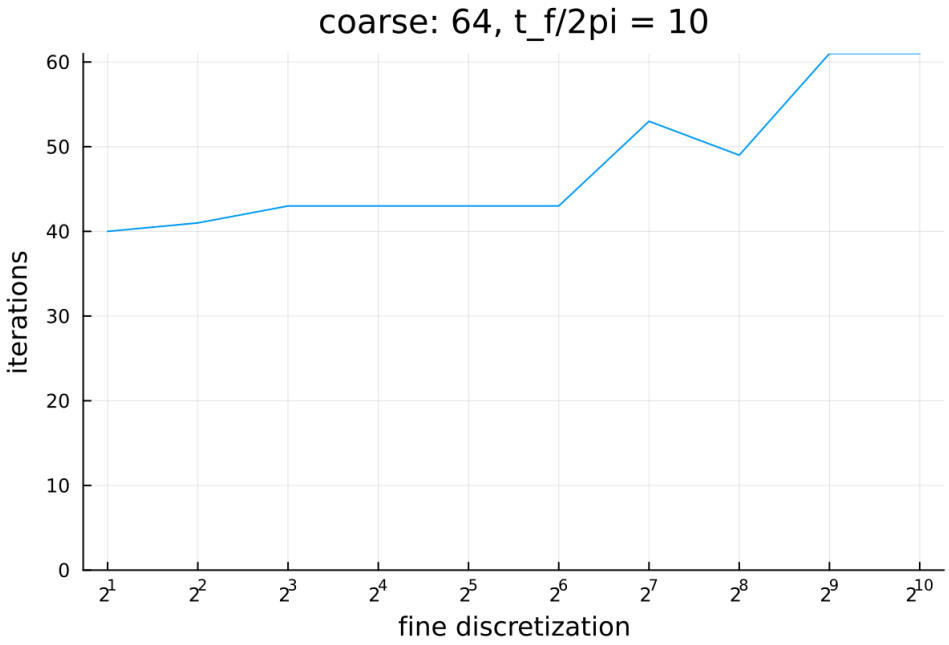

Figure 12:

The RoC of the PA has been well studied [33], [47]. Figure 11 shows this implementation converges linearly for small coarse discretizations, and superlinearly for large discretizations matching literature. Figure 12 shows the RoC of this implementation only slightly depends on the fine discretization for a small coarse discretization. Further analysis could be done for both a large coarse and fine discretizations. The results shown in this section provide substantial confidence that the behavior of further results will be consistent with those found in literature.

5.2. Data Transfers & Latency

As noted in Section 4.1, any data that is computed in one memory space must be transferred to another memory space in order for that data to be used in that space. Again, this applies for both GPUs and remote hosts (or more generally processes). Because these transfers happen over either PCI-E or network channels, respectively, they serve as the single greatest bottlenecks for performance in this implementation. The following discussion will address to what extent this implementation is limited by these bottlenecks and some strategies on how to mitigate them.

To highlight the bottlenecks in the PA, we focus our attention back to Algorithm 14. After the director has created the subproblems, it transfers them to each of the worker processes via a network communication. Then the director has to further communicate that the worker execute the PA on the GPU. The worker host then transfers the problem data to the worker device. After the GPU executes the Parareal kernel, the new data is transferred back to the host. Finally, the director requests the new data from the host, to which the worker transfers the new data over the network.

These transfer bottlenecks fall into two classifications: host-device, and host-host communication. There are several methods that can be used to mitigate the consequences of the host-device transfer, including taking advantage of dynamic parallelism (where the coarse propagation would be done on the device), pinned/page-locked memory, and zero-copy memory. As for host-host communication, which is done over the network and is the slowest part of the entire implementation, not much can be done to improve it directly other than using faster hardware and/or potentially an optimized cluster configuration, however, the relative efficiency could be improved by transferring as much data at once as possible.

5.2.1. Host-Device Data Transfers

A fundamental bottleneck, as it's part of the algorithm, is the need for all parallel computation to stop and the serial execution of the coarse propagation to complete. While the serial execution is itself a bottleneck in terms of performance, the coarse propagation requires the fine-propagation data on the device, and thus it must first be transferred to the host; similarly, the new data must be transferred to the device after it's been created. These transfers happen over PCI-E, which is really slow compared to the speeds of on-device memory transfers. In fact, the bandwidth of global memory on a device can be at least an order of magnitude greater than the PCI-E bandwidth [45].

These slow transfers could be completely circumvented by moving the coarse propagation to a single thread on the device, then using dynamic parallelism (where GPU kernels can be launched from within a GPU kernel so no CPU communication is required) to launch the Parareal kernel also on the device [48]. While a single device-thread is likely to take more time than a single host-thread when performing the same task, the increase in time from this trade is likely to be less than the decrease in time from not needing to transfer data. Thus the net time difference from this change would be beneficial. Though, this makes sense only when computing everything locally

as any distribution to other nodes would requires the use of the host system or NVIDIA's remote direct memory access (GPUDirect RDMA).

If the number of transfers is unchanged, the implementation could make use of pinned/page-locked memory. Pinned memory being memory on the host which the device can access directly without first requesting the host CPU to retrieve it and send it. Additionally, pinned memory is guaranteed to never be swapped out to disk and thus the device does not need to wait for it to first be transferred to pinned memory [45]. Using this technique could greatly improve performance; the implementation provided in this work does not use it but could be included via library calls [49], [50].

Another point that should be addressed is that the transfer rate between the host and device is not independent of the size of the transfer itself. On some devices, the transfer rate is nowhere near optimal when then the size of the transfer is below ~2 MB (even with pinned memory), and peak efficiency is only gained with transfers of 16 MB or more [45]. If the coarse discretization is equal to the number of threads the device can execute simultaneously, say $N_t = 5 * 10^3$, (i.e. there is a single problem for each device thread), and each problem needs and generates two 3-dimensional vectors (the position and velocity) composed of 32-bit floats ($S = 2 * 3 * 4$ bytes $= 24$ bytes), then each transfer into and out of the device would only consist of $N_t S = 120$ KB. As this is orders-of-magnitude smaller than what is needed for optimal efficiency, each thread of the device could instead operate on 16 MB /20 KB ≈ 133 problems!

If there are indeed multiple problems per device-thread, then zero-copy memory could be used to write a solution to global memory before the thread begins on the next one [45]. This way coarse propagation could begin while problems are still being fine-propagated on the device as shown in Figure 13. That being said, there is no guarantee the solutions needed in the coarse propagation will be written in the order they are needed.

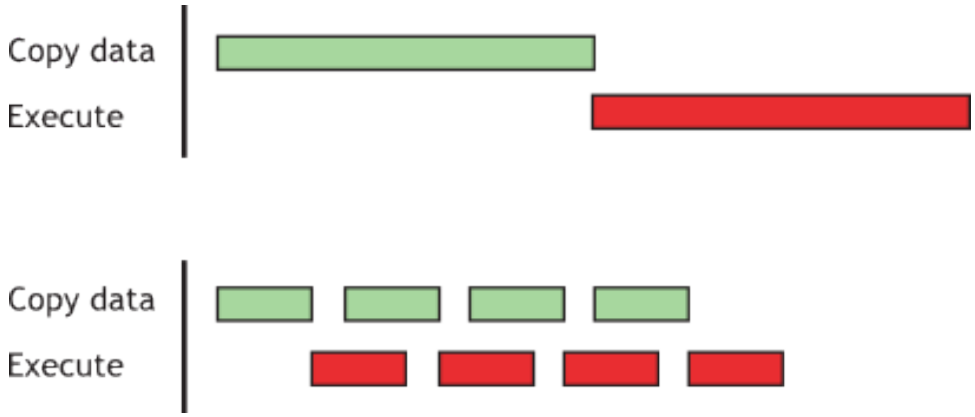

Figure 13: When only using pinned memory, all data needs to be copied from the host to the device before any computation can begin (top). Using zero-copy memory (bottom) allows for data transfers to happen at the same time as computation. Sourced from [51].

Overall, transfers can be avoided nearly completely by executing the coarse propagation on the device and distributing the problems to remote machines via RDMA. If problems are distributed, then the transfers would need to use pinned memory to avoid waiting for the CPU. In any case, the transfer-rate across PCI-E directly depends on the amount of data being transferred, so it's best to saturate not only all available threads on the device, but also the number of problems per thread. Even with optimally efficient data transfers, the PA is still bottlenecked by its sequential coarse propagation.

5.2.2. Host-Host Data Transfers

As described in Section 4, every time this implementation finishes coarse-propagating and creating the subproblems, those problems are distributed from the director to a worker nodes over the network. Not only does there need to be more work done in order to transmit the problems (e.g. the problems need to be converted from structured data to a bit-stream by serialization) but the throughput of networking hardware is much, much slower compared even to PCI-E.

It has been shown that cluster topology (i.e. how workers are related to each other, not necessarily physically) can have a significant effect on the performance of inter-machine communication [52]. As such, there are two topologies to consider: the network associated with the physical path any data takes (e.g. all data has to go through the director), and the network of nodes each node can "see". The former *physical topology* consists of the tangible material through which electrical impulses are sent such as Ethernet cabling. The latter *logical topology* encodes the worker that a particular worker can share data.

For example, the cluster shown in Figure 7 was designed to have the worker processes only communicate with the manager process on the same machine. This is so only the manager would need to send a single batch of data between physical machines to the director. While this logical topology seems sound, the actual path the data takes (according to the underlying physical topology) may be significantly detrimental to the overall performance of the cluster.

If the software that manages the message passing requires the director to act as an intermediary, then the physical topology of the cluster can have significant influence on its performance. What looks like a straightforward intra-machine communication in the logical topology, data moving between worker and manager, actually results in data being transferred from the worker process to the director. Then the data moves from the director to the manager to sit idle. Finally, the data moves from the manager back to the director, effectively creating a star topology as shown in Figure 14.

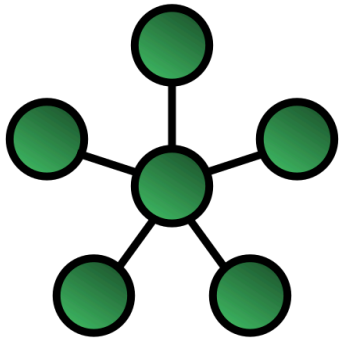

Figure 14: While a cluster might have the logical form as shown in Figure 7, the physical characteristics of the network connections and how the communication management controls the flow of data must still be considered. It is possible for the physical topology to be a star. Image sourced from [53].

Situations such as these can be mitigated in the future "simply" by only creating manager processes on the compute nodes, and having each device communicate with the host on its own dedicated thread. This way the device-manager-director trace truly is linear and efficient. Though, while this idea is simple in description, the implementation to have the manager process control multiple devices on different threads needs more care, but is still reasonable [50].

While distributed functionality is key to achieving scalability, the inter-process and inter-machine communication overhead is unavoidable, and thus so is its overhead. As long as care is taken when considering potentially hidden aspects of working with unshared memory spaces, such the physical versus logical topologies, this communication overhead can be minimized. Additional performance can be gained by minimizing the number of unshared memory spaces in general and opting for shared memory spaces such as multiple threads where no such overhead arises.

5.3. Benchmarks

Benchmarks and other performance evaluations are meaningless without the appropriate context into how the data was gathered. Just as an engineer should include the relevant model of their equipment in their reported data, if the computational scientist wishes their work to be reproducible, such information must be included with the data. As such, the hardware and software specifications that were used in these experiments is presented in Table 1. Further specifications on the GPUs that were used is collected in Table 2. Additionally, inter-node network traffic was routed through a TP-Link TL-SG108 1Gb/s network switch.

	Director	**Worker D**	**Worker L**
Operating System	Arch Linux (64-bit)	Arch Linux (64-bit)	Arch Linux (64-bit)
Kernel	6.14.10	6.14.9	6.15.9
Motherboard	ASRock B760M	ASUS H170	ThinkPad X1 Extreme Gen 3
CPU	Intel i7-13700K 24 logical cores @ 5.40 GHz	Intel i5-6500 4 logical cores @ 3.60 GHz	Intel i9-10885H 16 logical cores @ 5.30 GHz
GPU	NVIDIA RTX 3060 Ti	NVIDIA GTX 1660 Sup NVIDIA GTX 960	NVIDIA GTX 1650 Ti Mobile
Memory	32 GB DDR5 @ 6500 MHz	16 GB DDR4 @ 1600 MHz	16 GB DDR4 @ 2667 MHz

Table 1: Specifications for each host/node in the used cluster.

	Director	**Worker D**	**Worker D**	**Worker L**
Cores	4864	1408	1024	1024
Streaming Multiprocessors	38	22	8	16
Base Clock (MHz)	1410	1530	1176	1350
Boost Clock (MHz)	1665	1785	1201	1485
Memory Clock (Gb/s)	14	14	7	12
Memory Size (GB)	8	6	4	4
Memory Bandwidth (GB/s)	448	336	112	192
FP16, 32, 64 Performance (TFLOPS)	16.2, 16.2, 0.253	10.0, 5.03, 0.157	N/A, 2.46, 0.077	6.08, 3.04, 0.950
Compatible CUDA up to	8.6	7.5	5.2	7.5

Table 2: Hardware specifications for the GPUs used in this cluster.

On the software side, the Julia language was used to encode the calculations. The Julia standard library's Distributed.jl package was used to perform any and all distributed functionality. Additionally, the CUDA.jl package was used to facilitate the implementation of GPU-based calculations. The versions of these packages are detailed in Table 3.

Julia Runtime	1.11.5	LLVM	libLLVM-16.0.6
Distributed.jl	1.11.0	CUDA.jl	5.7.3

Table 3: The software versions used in the calculations presented in this work.

Figure 15 shows how the runtime of the simulation depends on the coarse and fine discretizations when using a purley sequential algorithm; these results are using the velocity-Verlet method. The symmetry these plots is consistent with the fact that in a sequential algorithm, the total discretization is effectively the product of the coarse and fine discretizations. Otherwise, both plots show that doublings of the discretization yield linear increases in the runtime.

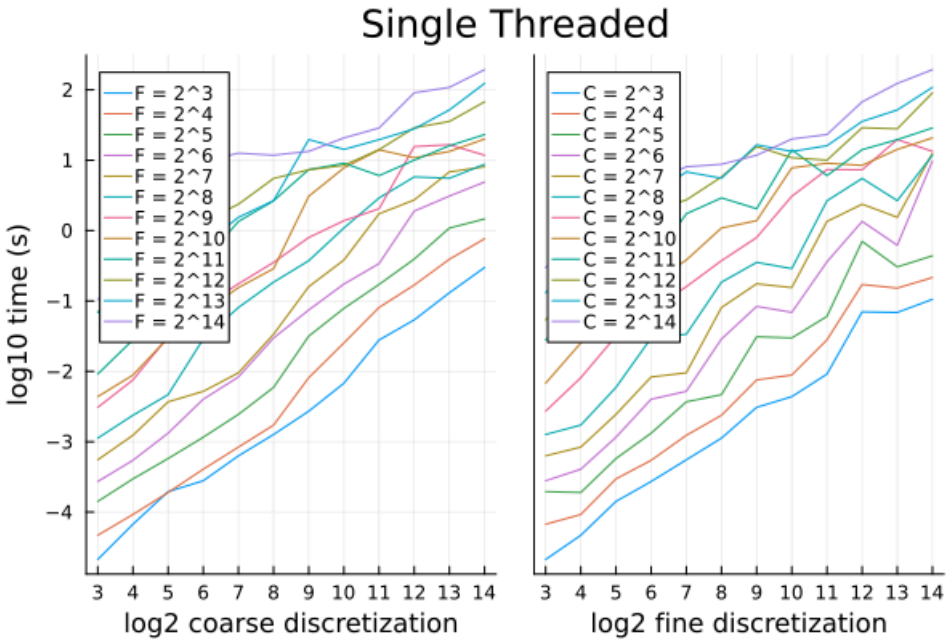

Figure 15: The runtime of a sequential integration algorithm depends effectively on the product of the coarse (left) and fine (right) discretizations.

Figure 16 shows how the runtime depends on the coarse and fine discretizations when using multithreading on the GPU. The dependence of the runtime's order of magnitude on the coarse discretization (left) is shown to be linear for all fine discretizations. More specifically, an increase of coarse discretization by a factor of eight leads to the runtime increasing by a factor of ten, approximately. Additionally, nearly all fine discretizations yield similar results meaning that the runtime is mostly insensitive to the fine discretization.

Continuing with Figure 16, the efficiency of the simulation when done using GPUs is investigated. The metric for this efficiency is considered to tbe the ratio between the products of error ε (as calculated in Section 5.1) and the runtime τ as $|\varepsilon\tau/\varepsilon_1\tau_1|$, where ε_1, τ_1 are the error and runtime of the sequential implementation for each coarse discretization i.e. the ratio is between error and runtime for the same coarse discretization. As the goal is to minimize both error and runtime, the goal is thus to minimize this metric. In other words, lower is better.

As can be seen in both the figures for the coarse (left) and fine (right) discretiation in Figure 16, the simulation becomes drastically inefficient. For a spectrum of coarse discretizations, most simulations behave similarly, but those with the largest fine discretizations diverge; the rate of this divergence is proportional to the fine discretization. This is consistent with the fact that GPUs will take the same amount of time to run any number of threads less than what it's capable of. In other words, small discretizations don't saturate the GPU and thus will all run in the same amount of time while the error decreases as the coarse discretization increases. For a spectrum of fine discretizations, the efficiency of the simulation is nearly independent of them.

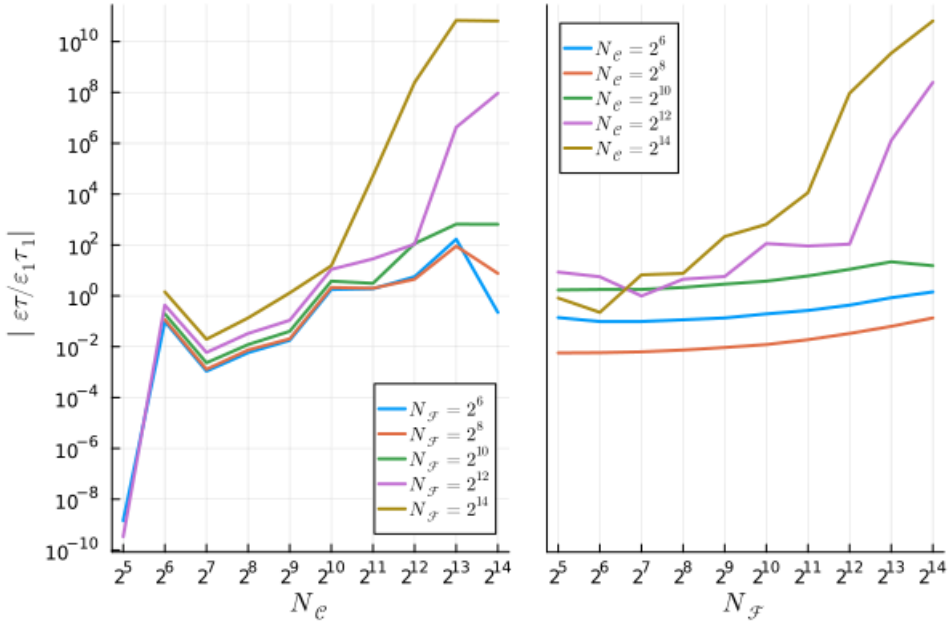

Figure 16: The efficiency of the simulation when run on a single GPU, characterized by the product of its error and runtime. This efficiency depends both on the coarse (left) and fine (right) discretizations.

Figure 17 shows how the runtime depends on the coarse and fine discretizations when using the distributed implementaion presented in this work. The overall behavior of these distributed results matches those of only using a single, local GPU. One notable difference between these and the single-GPU results is that for both large coarse and fine discretizations, the runtime for the distributed method is approximately an order of magnitude larger than that of the single-GPU implementation.

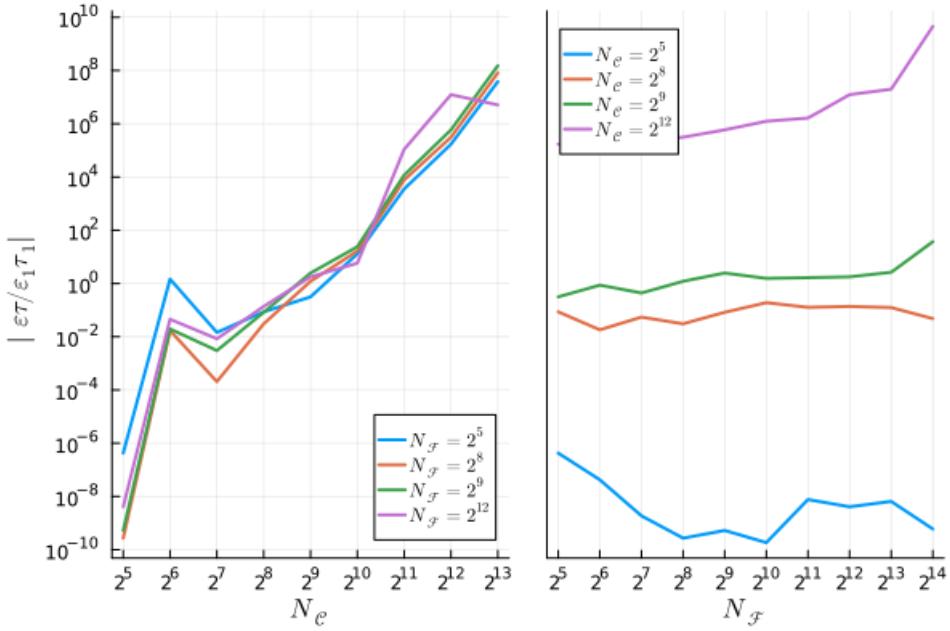

Figure 17: The efficiency of distributed simulations and how they depend on their coarse (left) and fine (right) discretizations.

Figure 18 compares the efficiencies between the different multithreading methods for both a range of coarse and fine discretizations. Over a spectrum of coarse discretizations (left), the GPU and distributed simulations achieve near identical behavior in their efficiencies. These trends only begin diverging significantly for a coarse discretization of 2^{10}. Likewise, the efficiency of the simulation for increasing fine discretizations (right) starts poorly for low discretization, but remains near its starting level until the a fine discretization of 2^{10}, just as with the coarse discretization.

The divergence seen in the previous figures occurs near a discretization of 2^{10}. This is mostl likely explained by round-off error becoming significant as when both discretizations are 2^{10}, the resulting time-step is (for this simulation) on the on the same order of magnitude as the precision of 32-bit floats. Because the representations used here are only accurate out to approximately seven decimal places, any difference smaller than that will yield incorrect results. As the error of the simulation only compounds with each step, using too large of discretizations, actually yields worse performance and less accurate results. Disappointingly, the simulations used here show that the distributed implementation does not offer increases in performance compared to executing the simulating with local-GPU nor even sequentially.

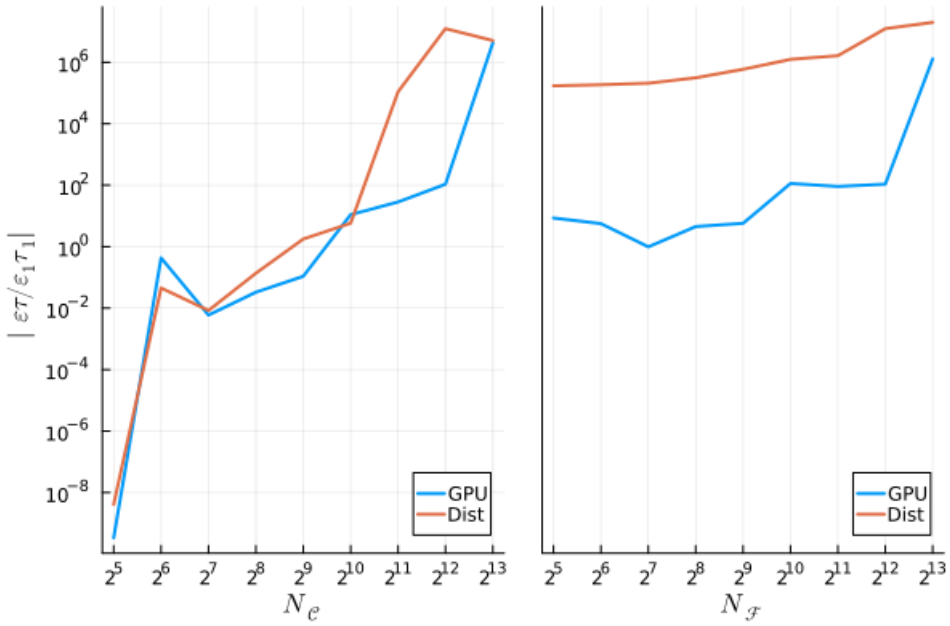

Figure 18: The runtime of the simulation depends on the method employed. The comparison of these runtimes is shown across the range of coarse (left) and fine (right) discretizations for the a both a fine and coarse discretization of 2^{12}, respectively, chosen to highlight the differences between the methods.

The most significant factors that influence the performance of this implementation are the latency from data-transfers, numerical instability due to using too small discretization, instability from significant round-off error due to using too small discretization, combined with using a low-precision representation of floating point numbers. Because communication occurs to frequently in the PA e.g. every iteration data transfers between hosts, the effects of data-transfer latency are only magnified to an extent proportional to the coarse discretization. While the PA is well-suited to take advantage of the massive parallelism of GPUs, the number of subproblems needed to make total use of the GPU requires discretizations that yield quantities that are sensitive to round-off error; this is only exacerbated by the standard of using only 32-bit precision representations of numbers. Similarly, because the number of subproblems to saturate the GPU is so large, the storage capacity needed for these subproblems also limits the effective coarse discretization and performance; such memory capacity is the reason why this work only considers discretiations up to 2^{14}. Systems with a larger memory capacity would be able to store enough subproblems that could also fill the GPU, resulting in simulations using this implementation that would out-perform resource constrained systems.

6. Conclusion

The Parareal Algorithm and other methods in the field of parallel-in-time integration are growing increasingly important and popular. Their use in computational science has seen significant increases in the past few years in the fields of physics, manufacturing, logistics, biology, and quantitative finance. This increase in popularity means there will be increased pressure for these methods to provide significant performance increases when simulating time-dependent systems, such as solving equations of motion in physics and molecular dynamics. While there are several implementations of the Parareal Algorithm, few have utilized techniques from high-performance computing, and those that do, rely on implementations developed with legacy methods. The aim of this work was to provide an implementation of the Parareal Algorithm to efficiently solve equations of motion for even the biggest problems while bringing computational science to the modern age.

The Parareal Algorithm itself divides equations of motion into many smaller problems in order to conquer them simultaneously. After the smaller problems are solved, their solutions are combined to recreate the problems with increasingly accurate guesses for their initial values. This process iterates until the solution of the root problem is the same as what would be produced by directly using an accurate traditional method. Because this process results in many independent problems, its performance is theoretically only limited by the number of threads it has available.

Because the Parareal Algorithm wraps around traditional numerical solvers that only use arithmetic operations, it is well suited to take advantage of the massive parallelism offered by graphics processing units. Though, even more parallelism can be gained by connecting multiple machines together in order to harness their clustered computational power. These architectures together theoretically offer unlimited computational power so that the PA can scale to whatever science needs to be done.

This work has shown a scalable version of the PA can produce results with numerical accuracy and behavior similar to those found in literature. While there is significant latency associated with transferring data between memory spaces, whether it be between hardware components local to a single machine or between physically separated machines, methods to mitigate or even circumvent these latencies, such as pinned-memory and cluster-topology optimization, have been developed and can be employed in future investigations. Finally, benchmarks show that the current implementation of a distributed, GPU-based PA does not increase performance compared to using a local GPU or even a single threaded approach; this ranking could change for discretizations larger than those measured here.

6.1. Future Work & Possible Optimizations

This work focused on implementing the PA in the Julia programming language targeting CUDA-based GPUs with the Distributed.jl standard-library's support for multiprocessing and distributed computing; each of these specifications has alternatives. While one of the goals of this research was to implement the PA in the Julia language, the same functionality could be implemented using other languages and libraries. Additionally, because CUDA only works with NVIDIA GPUs, devices from other manufacturers are incompatible with this implementation and thus its availability is limited. For distributed functionality, there are several tried-and-true/de-facto standards that could be used as well.

A traditional HPC language such as C, C++, or Fortran could be used in conjunction with low-, and high-level accelerating libraries to theoretically increase the "raw" performance of the implementation. In reality, any potential performance gains would only arise if the manually-optimized, low-level source-code was more performant than the automatically-optimized, high-level source code i.e. "C is faster than Python, but is *your* C faster than Python?". For example, OpenMP could be used for high-level CPU-based multithreading, OpenACC could be used for high-level GPU-based multithreading, and MPI could be used for high-level distributed functionality. For further GPU optimization, directly using a lower-level GPGPU interface could be used based on the vendor of the targeted device: CUDA for NVIDIA GPUs, ROCm for AMD GPUs, Metal for Apple GPUs, oneAPI for Intel GPUs, and OpenCL or SYCL for platform agnostic implementations; while there are other interfaces that would allow for arbitrary computations to be done on the device, such as Vulkan and OpenGL, these are aimed more toward graphics processing. That being said, there has been and there continues to be much work on the KernelAbstractions.jl Julia package that provides functionality for source code to be not just vendor agnostic, but also heterogenous allowing the same source code to target both CPUs and GPUs.

In general, memory allocations should be minimized as they significantly reduce performance in both time and memory usage. This implementation creates a new subproblem each time, so modifying the method to adjust the subproblem in-place would drastically reduce the number of memory allocations. Additionally, custom types were used to facilitate source-code readability, but lighter data structures (e.g. arrays) could be used throughout to both minimize memory allocations, as well as allow for further automatic optimization from the compiler. These are merely a few observations of how this implementation could be optimized; there are certainly more that will be identified and addressed in the near future.

Bibliography

[1] github.com/Parallel-in-Time, Accessed: Jul. 16, 2025. [Online]. Available: https://parallel-in-time.org/

[2] J.-L. Lions, Y. Maday, and G. Turinici, "Résolution d'EDP par un schéma en temps «pararéel »," *Comptes Rendus de l'Académie des Sciences - Series I - Mathematics*, vol. 332, no. 7, pp. 661–668, 2001, doi: https://doi.org/10.1016/S0764-4442(00)01793-6.

[3] C. Farhat and M. Chandesris, "Time-decomposed parallel time-integrators: theory and feasibility studies for fluid, structure, and fluid-structure applications," *International Journal for Numerical Methods in Engineering*, vol. 58, no. 9, pp. 1397–1434, 2003, doi: 10.1002/nme.860.

[4] M. Emmett and M. L. Minion, "Toward an Efficient Parallel in Time Method for Partial Differential Equations," *Communications in Applied Mathematics and Computational Science*, vol. 7, pp. 105–132, 2012, doi: 10.2140/camcos.2012.7.105.

[5] D. Ruprecht, R. Speck, M. Emmett, M. Bolten, and R. Krause, "Poster: Extreme-scale space-time parallelism," in *Proceedings of the 2013 Conference on High Performance Computing Networking, Storage and Analysis Companion*, in SC '13 Companion. Denver, Colorado, USA, 2013. [Online]. Available: http://sc13.supercomputing.org/sites/default/files/PostersArchive/tech_posters/post148s2-file3.pdf

[6] A. J. Christlieb, C. B. Macdonald, and B. W. Ong, "Parallel high-order integrators," *SIAM Journal on Scientific Computing*, vol. 32, no. 2, pp. 818–835, 2010, doi: 10.1137/09075740X.

[7] Y. Maday and E. M. Rønquist, "Parallelization in time through tensor-product space-time solvers," *Comptes Rendus Mathematique*, vol. 346, no. 1–2, pp. 113–118, 2008, doi: 10.1016/j.crma.2007.09.012.

[8] M. J. Gander, J. Liu, S.-L. Wu, X. Yue, and T. Zhou, "ParaDiag: parallel-in-time algorithms based on the diagonalization technique," 2021. [Online]. Available: http://arxiv.org/abs/2005.09158

[9] G. Horton and S. Vandewalle, "A Space-Time Multigrid Method for Parabolic Partial Differential Equations," *SIAM Journal on Scientific Computing*, vol. 16, no. 4, pp. 848–864, 1995, doi: 10.1137/0916050.

[10] C. Lubich and A. Ostermann, "Multi-grid dynamic iteration for parabolic equations," *BIT Numerical Mathematics*, vol. 27, no. 2, pp. 216–234, 1987, doi: 10.1007/BF01934186.

[11] S. G. Vandewalle and E. F. Van de Velde, "Space-time concurrent multigrid waveform relaxation," *Annals of Numerical Mathematics*, vol. 1, no. 1–4, pp. 347–360, 1994, doi: 10.13140/2.1.1146.1761.

[12] G. Horton, S. Vandewalle, and P. Worley, "An Algorithm with Polylog Parallel Complexity for Solving Parabolic Partial Differential Equations," *SIAM Journal on Scientific Computing*, vol. 16, no. 3, pp. 531–541, 1995, doi: 10.1137/0916034.

[13] S. Vandewalle and G. Horton, "Fourier mode analysis of the multigrid waveform relaxation and time-parallel multigrid methods," *Computing*, vol. 54, no. 4, pp. 317–330, 1995, doi: 10.1007/BF02238230.

[14] S. Friedhoff, R. Falgout, T. Kolev, S. P. MacLachlan, and J. B. Schroder, "A Multigrid-in-Time Algorithm for Solving Evolution Equations in Parallel," in *Presented at: Sixteenth Copper Mountain Conference on Multigrid Methods, Copper Mountain, CO, United States, Mar 17 - Mar 22, 2013*, 2013. [Online]. Available: http://www.osti.gov/scitech/servlets/purl/1073108

[15] D. Ruprecht, "Shared Memory Pipelined Parareal," in *Euro-Par 2017: Parallel Processing: 23rd International Conference on Parallel and Distributed Computing, Santiago de Compostela, Spain, August 28 – September 1, 2017, Proceedings*, F. F. Rivera, T. F. Pena, and J. C. Cabaleiro, Eds., Springer International Publishing, 2017, pp. 669–681. doi: 10.1007/978-3-319-64203-1_48.

[16] M. Schreiber, A. Peddle, T. Haut, and B. Wingate, "A Decentralized Parallelization-in-Time Approach with Parareal." [Online]. Available: https://arxiv.org/abs/1506.05157

[17] T. M. Masthay and S. Perugini, "Parareal Algorithm Implementation and Simulation in Julia." [Online]. Available: https://arxiv.org/abs/1706.08569

[18] R. Speck *et al.*, "Parallel-in-Time/pySDC." [Online]. Available: https://github.com/Parallel-in-Time/pySDC

[19] R. Speck, "Algorithm 997: pySDC—Prototyping Spectral Deferred Corrections," *ACM Trans. Math. Softw.*, vol. 45, no. 3, Aug. 2019, doi: 10.1145/3310410.

[20] J. Hahne, S. Friedhoff, and M. Bolten, "PyMGRIT: A Python Package for the parallel-in-time method MGRIT," 2020. [Online]. Available: http://arxiv.org/abs/2008.05172v1

[21] "XBraid: Parallel multigrid in time."

[22] S. Särkkä and Á. F. García-Fernández, "Temporal Parallelization of the HJB Equation and Continuous-Time Linear Quadratic Control," *IEEE Transactions on Automatic Control*, vol. 70, no. 6, pp. 3755–3770, Jun. 2025, doi: 10.1109/tac.2024.3518309.

[23] S. Pamela *et al.*, "Neural-Parareal: Self-improving acceleration of fusion MHD simulations using time-parallelisation and neural operators," *Computer Physics Communications*, vol. 307, p. 109391, Feb. 2025, doi: 10.1016/j.cpc.2024.109391.

[24] I. Jimenez-Ciga, A. Arrarás, F. J. Gaspar, and L. Portero, "Space-Time Parallel Parareal Algorithms for Pattern Formation Models," in *Numerical Mathematics and Advanced Applications ENUMATH 2023, Volume 2*, Springer Nature Switzerland, 2025, pp. 1–11. doi: 10.1007/978-3-031-86169-7_1.

[25] A. Q. Ibrahim, S. Götschel, and D. Ruprecht, "Space-Time Parallel Scaling of Parareal with a Physics-Informed Fourier Neural Operator Coarse Propagator Applied to the Black-Scholes Equation," in *Proceedings of the Platform for Advanced Scientific Computing Conference*, in PASC '25. ACM, Jun. 2025, pp. 1–11. doi: 10.1145/3732775.3733574.

[26] J. Barnes and P. Hut, "A hierarchical O(N log N) force-calculation algorithm," *Nature*, vol. 324, no. 6096, pp. 446–449, Dec. 1986, doi: 10.1038/324446a0.

[27] T. Hamada *et al.*, "A novel multiple-walk parallel algorithm for the Barnes–Hut treecode on GPUs – towardscost effective, high performance N-body simulation," *Computer Science - Research and Development*, vol. 24, no. 1, pp. 21–31, Sep. 2009, doi: 10.1007/s00450-009-0089-1.

[28] L. D. Landau and E. M. Lifshitz, *Mechanics, Third Edition: Volume 1 (Course of Theoretical Physics)*, 3rd ed. Butterworth-Heinemann, 1976. [Online]. Available: http://www.worldcat.org/isbn/0750628960

[29] S. A. Orszag, "Numerical Methods for the Simulation of Turbulence," *The Physics of Fluids*, vol. 12, no. 12, p. II–250–II–257, 1969, doi: 10.1063/1.1692445.

[30] C. R. (https://scicomp.stackexchange.com/users/18981/chris-rackauckas), "What does "symplectic" mean in reference to numerical integrators, and does SciPy's odeint use them?." [Online]. Available: https://scicomp.stackexchange.com/q/29154

[31] C. Cramer, *Essentials of Computational Chemistry: Theories and Models*. Wiley, 2013. [Online]. Available: https://books.google.com/books?id=k4R6cf7I7q0C

[32] H. Bock and K. Plitt, "A Multiple Shooting Algorithm for Direct Solution of Optimal Control Problems*," *IFAC Proceedings Volumes*, vol. 17, no. 2, pp. 1603–1608, 1984, doi: https://doi.org/10.1016/S1474-6670(17)61205-9.

[33] M. J. Gander and S. Vandewalle, "Analysis of the Parareal Time-Parallel Time-Integration Method," *SIAM Journal on Scientific Computing*, vol. 29, no. 2, pp. 556–578, 2007, doi: 10.1137/05064607X.

[34] C. Pinter, *A Book of Set Theory*. Dover Publications, 2014. [Online]. Available: https://books.google.com/books?id=iUT_AwAAQBAJ

[35] M. Harris, "CUDA Pro Tip: Write Flexible Kernels with Grid-Stride Loops." Accessed: Mar. 30, 2025. [Online]. Available: https://developer.nvidia.com/blog/cuda-pro-tip-write-flexible-kernels-grid-stride-loops/

[36] B. W. Ong and R. J. Spiteri, "Deferred Correction Methods for Ordinary Differential Equations," *Journal of Scientific Computing*, vol. 83, no. 3, p. 60, Jun. 2020, doi: 10.1007/s10915-020-01235-8.

[37] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms, fourth edition*. MIT Press, 2022. [Online]. Available: https://books.google.com/books?id=RSMuEAAAQBAJ

[38] N. Giordano and H. Nakanishi, *Computational Physics*. Pearson/Prentice Hall, 2006. [Online]. Available: https://books.google.com/books?id=52J6QgAACAAJ

[39] N. Chapman, "PararealGPU.jl." [Online]. Available: https://github.com/NonDairyNeutrino/PararealGPU.jl

[40] M. Harris, "How to Optimize Data Transfers in CUDA C/C++." Accessed: May 10, 2025. [Online]. Available: https://developer.nvidia.com/blog/how-optimize-data-transfers-cuda-cc/

[41] M. Harris, "An Even Easier Introduction to CUDA." Accessed: Apr. 17, 2025. [Online]. Available: https://developer.nvidia.com/blog/even-easier-introduction-cuda/

[42] D. Singal, "N Ways to SAXPY: Demonstrating the Breadth of GPU Programming Options." Accessed: Apr. 17, 2025. [Online]. Available: https://developer.nvidia.com/blog/n-ways-to-saxpy-demonstrating-the-breadth-of-gpu-programming-options/

[43] W. Stallings, *Operating Systems: Internals and Design Principles*. Pearson Education, 2011. [Online]. Available: https://books.google.com/books?id=bZwuAAAAQBAJ

[44] C. Li, C. Ding, and K. Shen, "Quantifying the cost of context switch," in *Proceedings of the 2007 Workshop on Experimental Computer Science*, in ExpCS '07. San Diego, California: Association for Computing Machinery, 2007, p. 2–es. doi: 10.1145/1281700.1281702.

[45] S. Cook, *CUDA Programming: A Developer's Guide to Parallel Computing with GPUs*, 1st ed. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 2012.

[46] J. Nocedal and S. Wright, *Numerical Optimization*. in Springer Series in Operations Research and Financial Engineering. Springer New York, 2000. [Online]. Available: https://books.google.com/books?id=w1kJYpOkPykC

[47] M. J. Gander and S. Vandewalle, "On the Superlinear and Linear Convergence of the Parareal Algorithm," in *Domain Decomposition Methods in Science and Engineering XVI*, O. B. Widlund and D. E. Keyes, Eds., Berlin, Heidelberg: Springer Berlin Heidelberg, 2007, pp. 291–298.

[48] A. Adinets, "CUDA Dynamic Parallelism API and Principles." Accessed: Jul. 03, 2025. [Online]. Available: https://developer.nvidia.com/blog/cuda-dynamic-parallelism-api-principles/

[49] T. Besard, C. Foket, and B. De Sutter, "Effective Extensible Programming: Unleashing Julia on GPUs," *IEEE Transactions on Parallel and Distributed Systems*, 2018, doi: 10.1109/TPDS.2018.2872064.

[50] T. Besard, V. Churavy, A. Edelman, and B. De Sutter, "Rapid software prototyping for heterogeneous and distributed platforms," *Advances in Engineering Software*, vol. 132, pp. 29–46, 2019.

[51] NVIDIA, "CUDA C++ Best Practices Guide." Accessed: Jul. 12, 2025. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/

[52] Y. Deng, M. Guo, A. F. Ramos, X. Huang, Z. Xu, and W. Liu, "Optimal low-latency network topologies for cluster performance enhancement," *The Journal of Supercomputing*, vol. 76, no. 12, pp. 9558–9584, Dec. 2020, doi: 10.1007/s11227-020-03216-y.

[53] WDGraham, "starNetwork." [Online]. Available: https://commons.wikimedia.org/wiki/File:StarNetwork.svg